

COMP1004-DBI

Databases and Interfaces

Module conveners:

Kai Xu, Nazia Hameed

Course practicalities

COMP1004-DBI

Module personnel

Convenor

Nazia Hameed- Room C72

Kai Xu- Room B83

Teaching support:

Ifeoma Ogbodo

Jude Afana

Damla Klick

Mahfuzah Fariha

yu ZHAO

Teaching overview (1)

- 10 credit Module = 100 hours of study time
- ~11 Weeks of Teaching
- Contact time:
 - Lectures (~22hrs)
 - Labs / practicals (22hrs)
 - Announcement on Moodle

Teaching overview (2)

- Each week:
 - Lecture (2hr) – in-person – Tuesday 11:00 am BS Auditorium
 - Exercises and worksheets (Moodle)
 - Labs (2hr) – Tuesday 2:00 pm A32/A07
 - Some weeks: Exercises and worksheets
 - **Specific lab: In-Lab test to be done in lab time slot (24th February 20206)**
- Watch for announcements in Moodle

Teaching overview (3)

- This module has two tracks:
 - ‘Theory’: database theory e.g., relational model, entity relationship diagram normalisation, SQL etc.
 - ‘Implementation’: SQL,HTML/CSS/JS,APIs etc.

Assessment

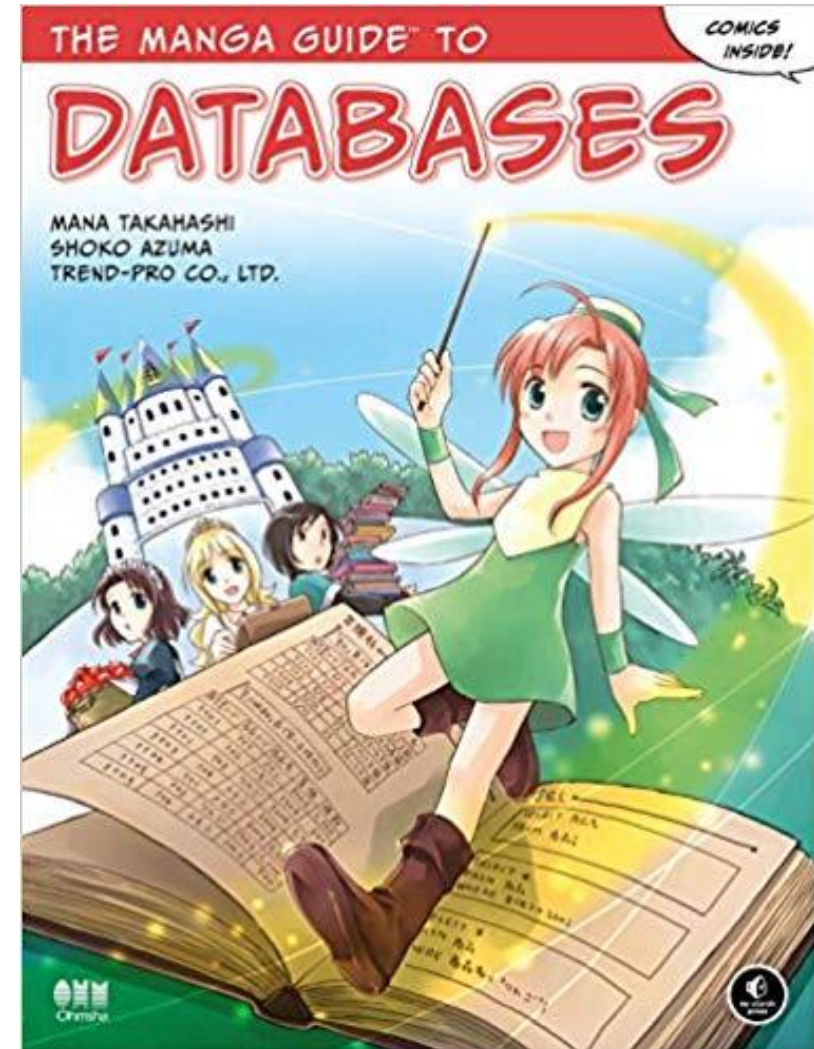
- In-Lab Test: total 20%
 - SQL(Moodle Quiz)
 - **Assessment will be conducted on Moodle during 2hr lab slots – 24th February 2026**
- Coursework: Project coursework – 50%
 - Database + web interface
- Exam:30%
 - Topics: Content cover in database

Learning objectives

- When you have finished this module you should be able to:
 - Conduct a requirements analysis for a database
 - Draw ER diagram
 - Design a data storage solution using a RDBMS
 - Use SQLite
 - Build and query a relational database using SQL
 - Design a web-based front-end (HTML, CSS, JS)
 - Integrate front-end with a RDBMS using backend
- Complete end-to-end web solution for any given case study

Resources

- Jeffrey A. Hoffer, V. Ramesh & Heikki Topi, Modern Database Management, Global Edition
- Connolly, T. M., and Begg, C. E., *Database Systems: A Practical Approach to Design, Implementation, and Management*
- [DB Browser for SQLite](#)



What is a database?

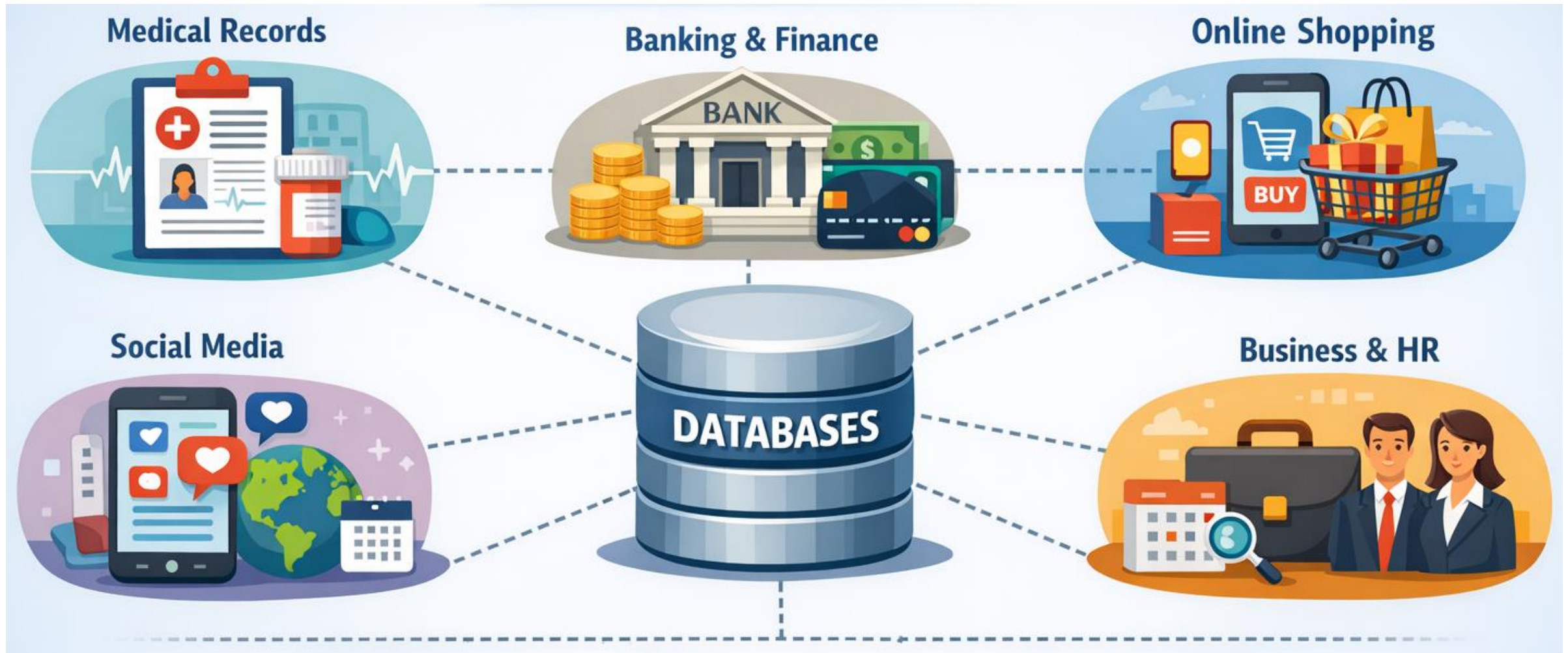
- Oxford English Dictionary
 - “A set of information held by a computer”
- Free On-Line Dictionary of Computing
 - “One or more large structured sets of persistent data, usually associated with software to update and query the data”
- American Heritage Science Dictionary
 - “A collection of data arranged for ease and speed of search and retrieval by a

Databases have been both essential and dull for millennia



Photo: Wikimedia/Alexander Paukner - Hieroglyphic number writing, 3rd millennium BCE, Karnak temple complex in Luxor

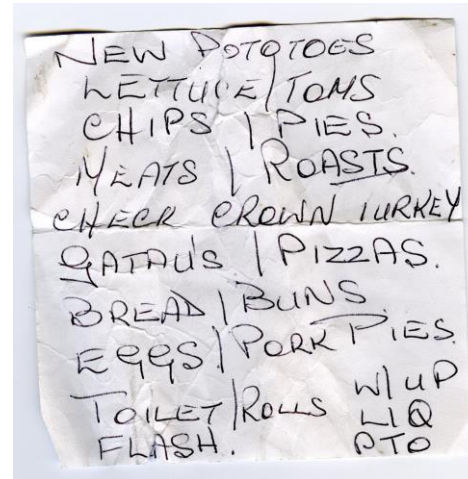
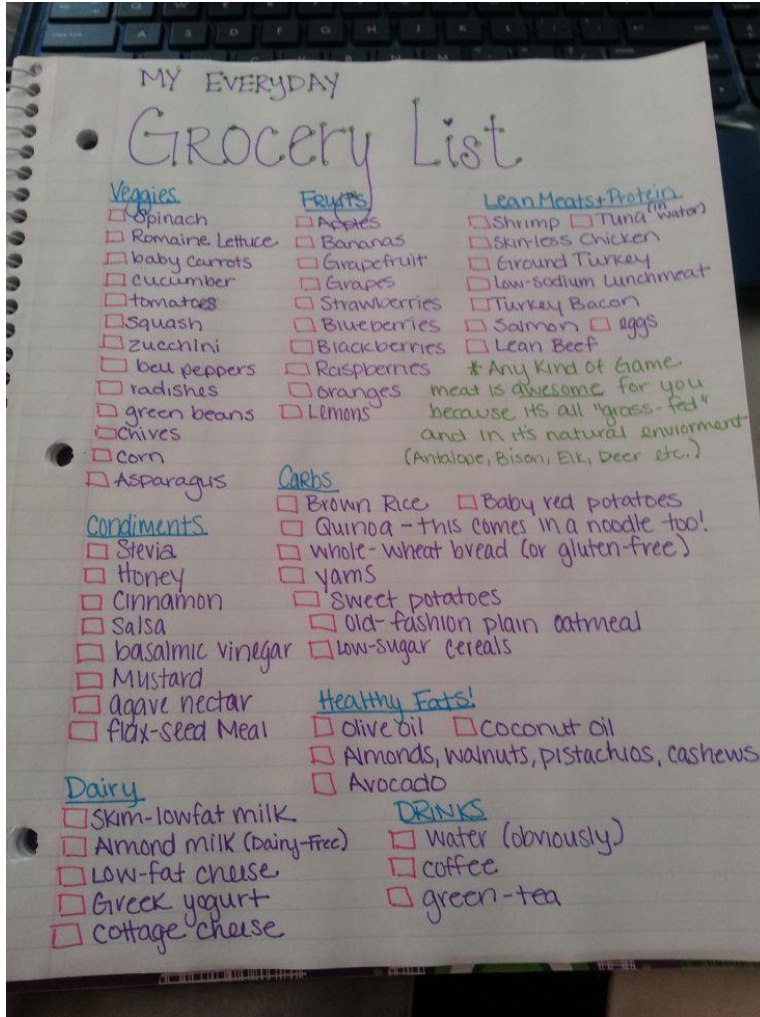
Uses of databases



Why study databases?

- Database systems give a set of tools for storing, searching and managing this information
- Databases are a **core topic** in computer science/data science
- Almost everything we see on a computer uses a DBMS or form of databases
- Basic concepts and skills with database systems are part of the skill set you will be assumed to have as a CS or data science graduate

Shopping list



- Informal organisation / complexity
- Handwriting

Medical records

ProgressEvaluation-V1.00-0083-TRE-CHOSP.doc

Version No1.0

Revision Date01/06/08

Review Date01/06/09

Derbyshire County Primary Care Trust

HOSPITAL IN-PATIENT MULTIDISCIPLINARY RECORDS – PROGRESS/EVALUATION

JRNAME

HOSPITAL

DRENAME

WARD

DDRESS

D.O.B

GP/Consultant

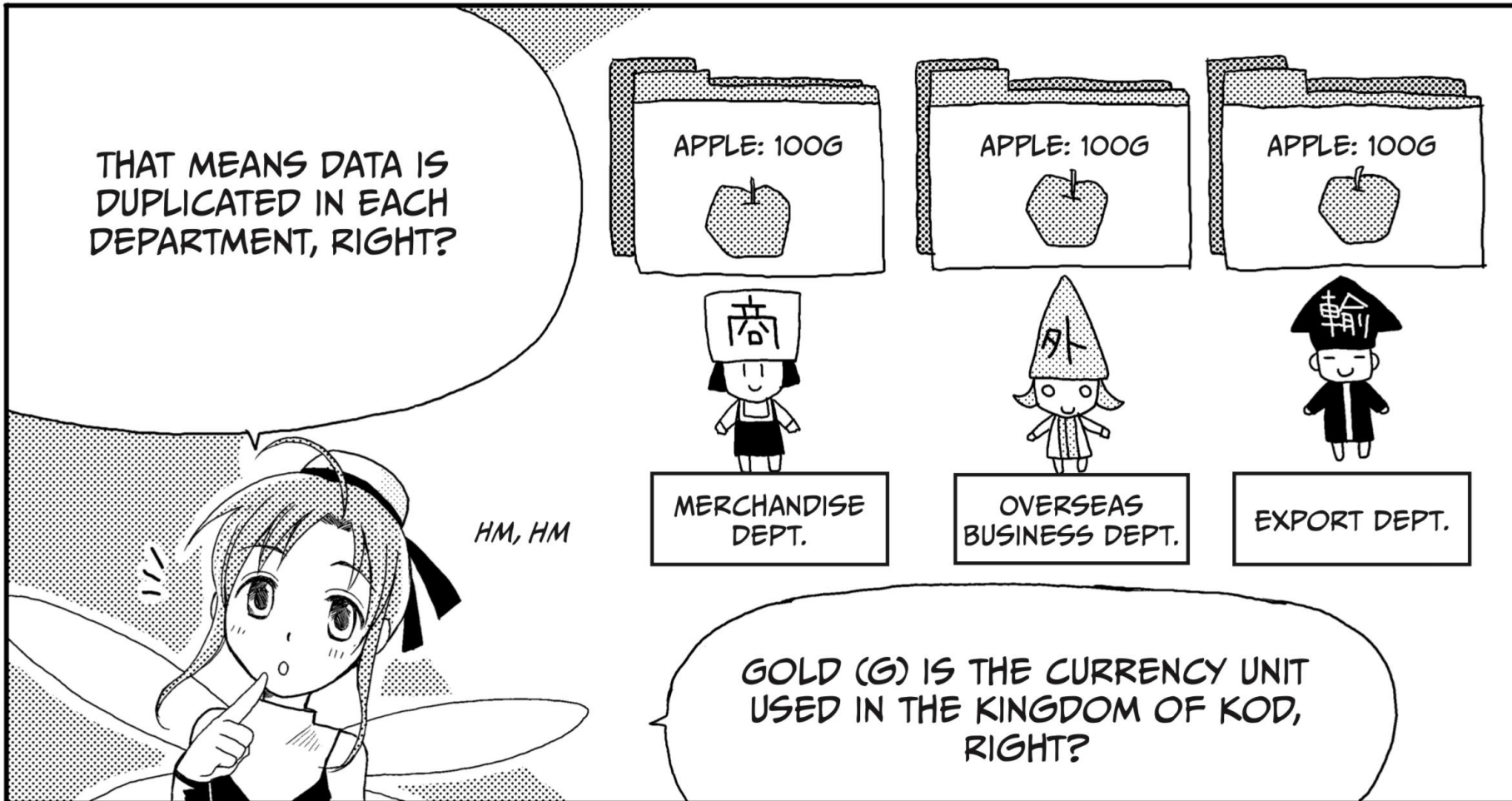
Date and time	Treatment Plan No	Comments / Details	Initials
13/06/08		Refused Soma at 2200hrs. Paracetamol not given due to 1500hrs dose being late as just arrived on ward.	
03/06/08		Both leg swaddles - skin at risk. Transferring with Rebecca + Z. Very deaf - needs constant prompts as to what Jade.	
21/06/08		HxH needed - on RHEMT leucocytes blood plus protein present	
27/06/08		Assisted with hygiene and dressing. Groins and sacral area sore, zinc and castor oil cream applied.	
28/06/08		Refused all laxatives this morning.	
30/06/08		Only ate small amount of soup and "picnics" at 01e sandwiches. To start food chart for 3 days to assess if eating well.	
30/06/08		Spoke to daughter who says her mum has a poor appetite normally and 01e supplements started at 0100PM	

Page 1 of 2

- Formal organisation (fields, rows, etc.)
- Handwriting
- Fields ('mis')used

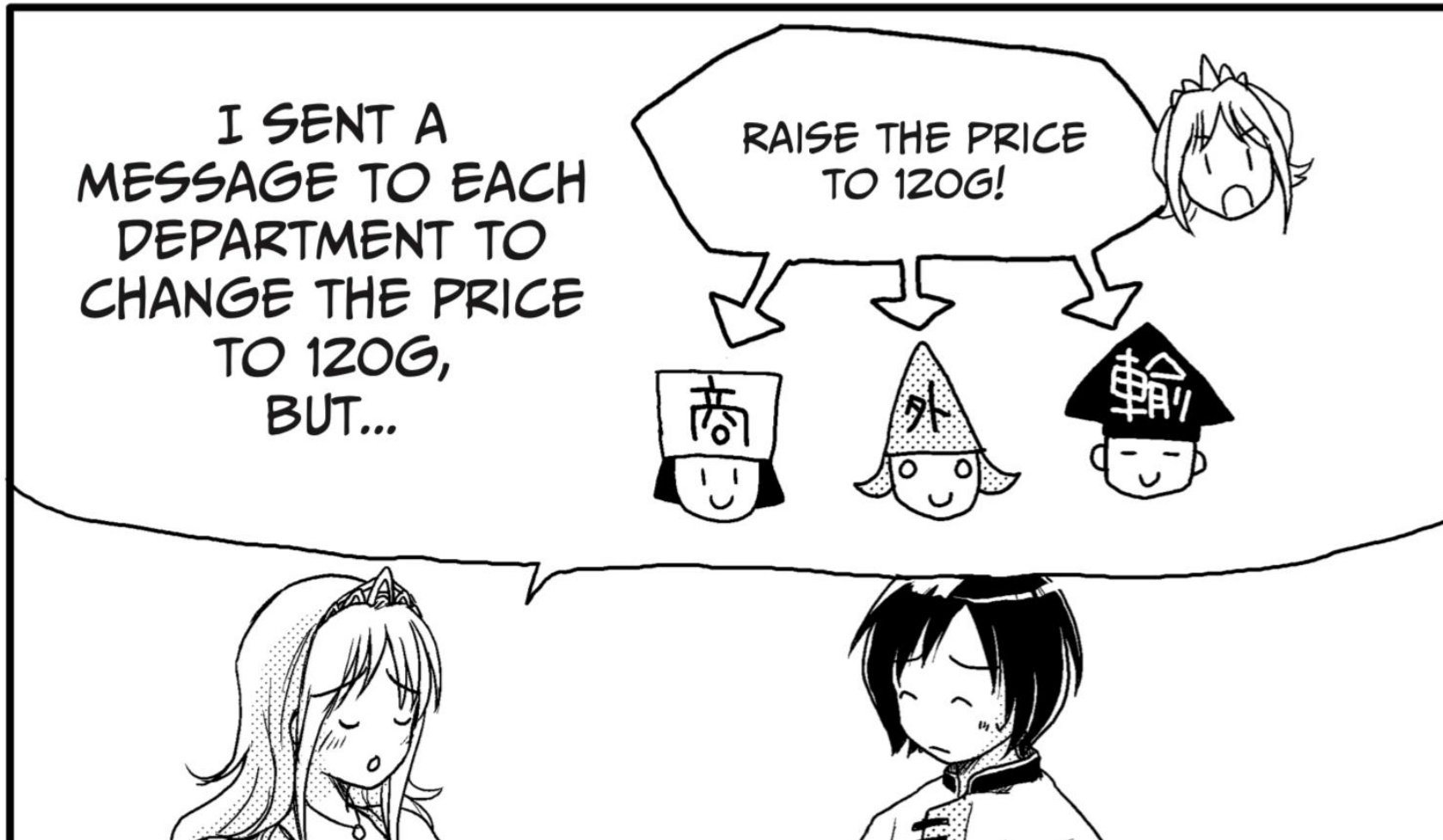
File-based databases: An example (1)

- From the *Manga Guide to Databases*
- The Kingdom of Kod exports apples (p. 11)



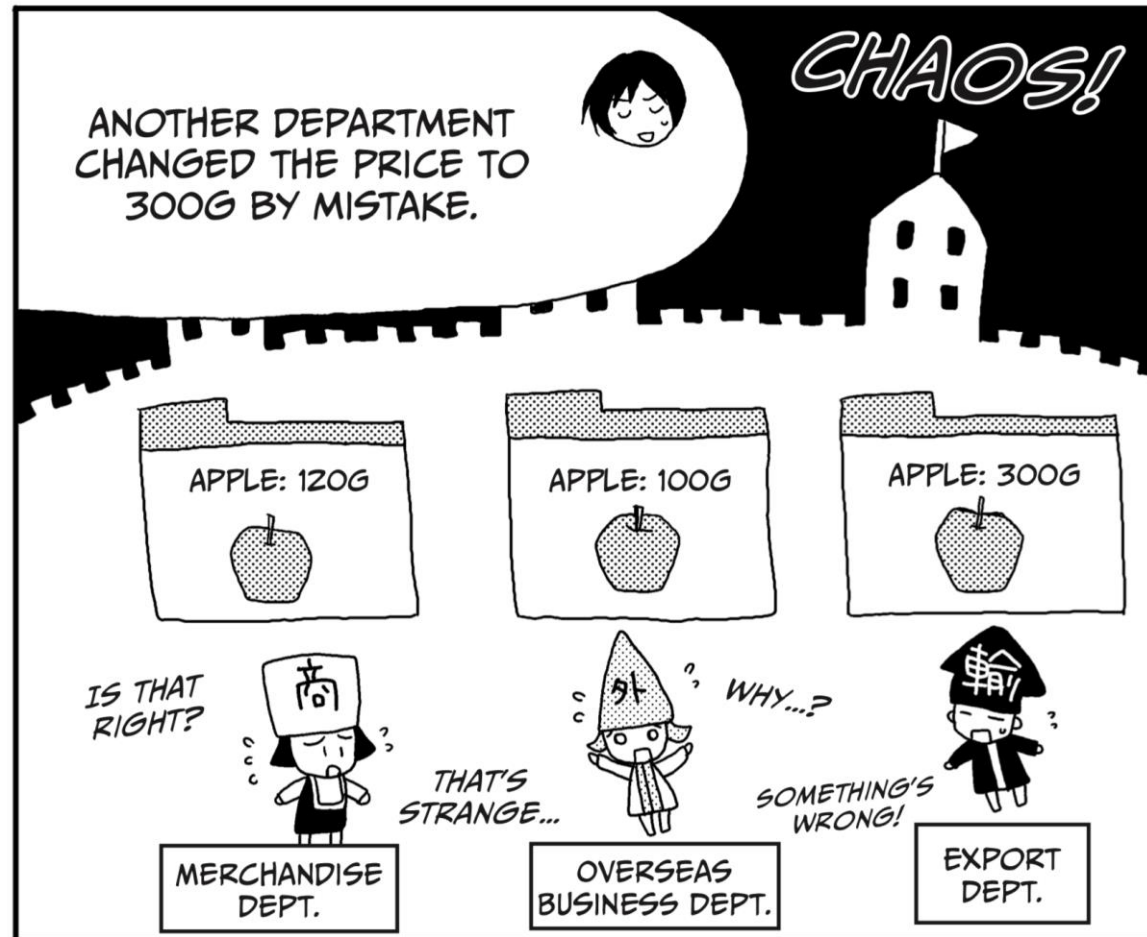
File-based databases: An example (2)

- What if the price changes?



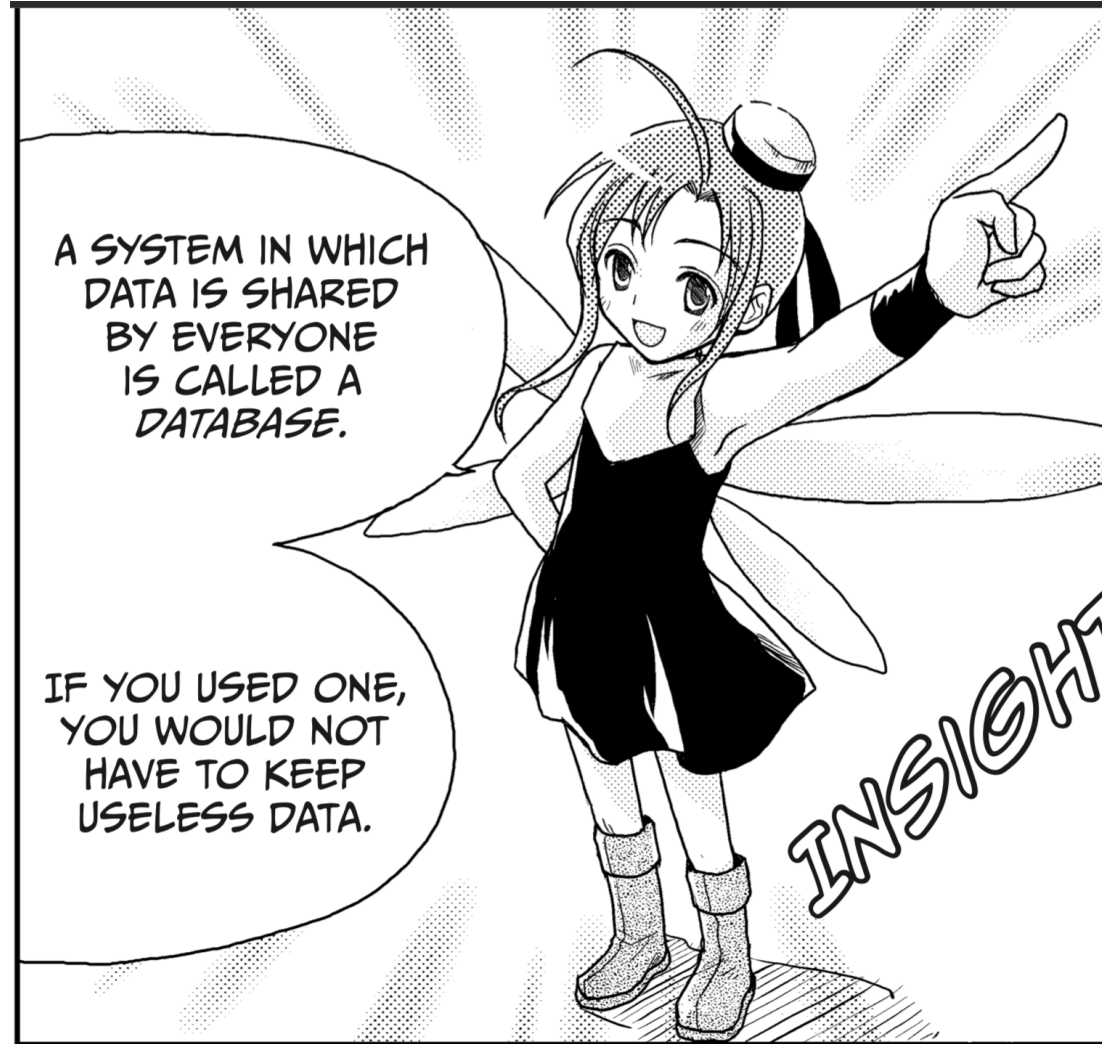
File-based databases: An example (3)

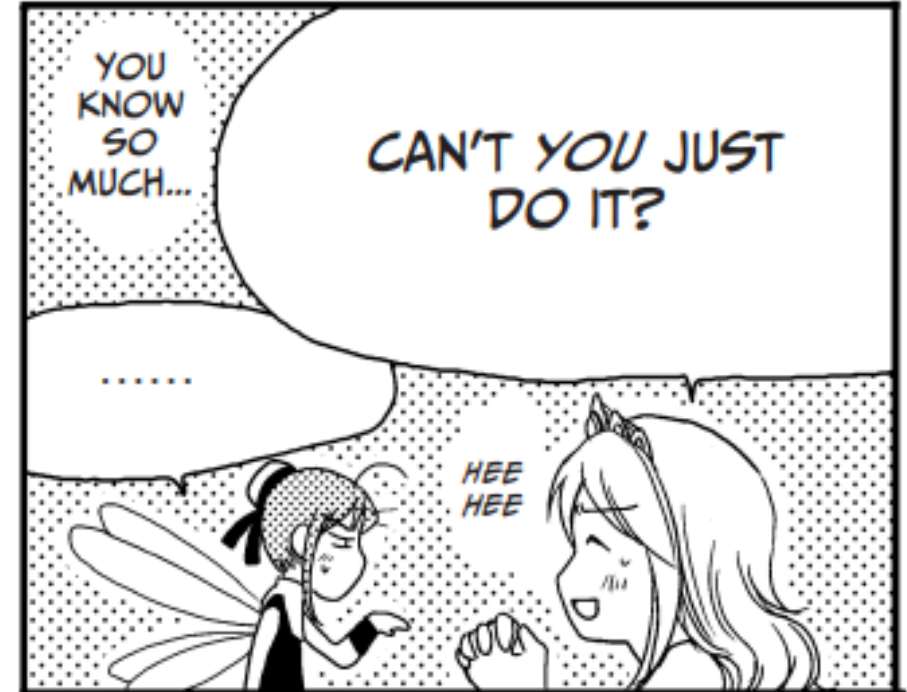
- Mistakes get made:
 - The overseas business department didn't get the message
 - The export department misheard the message



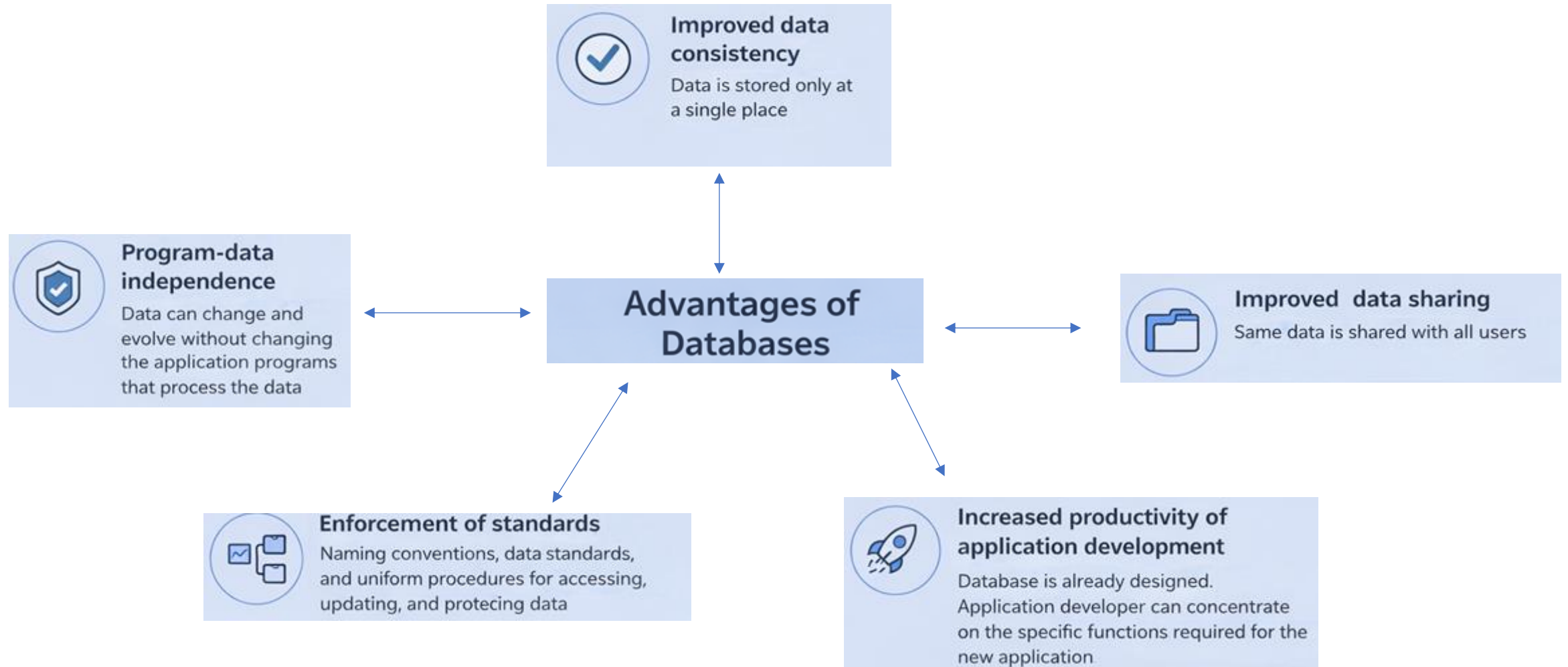
Redundant data is bad

- Storing the same data several times in different places (redundancy) is error-prone!





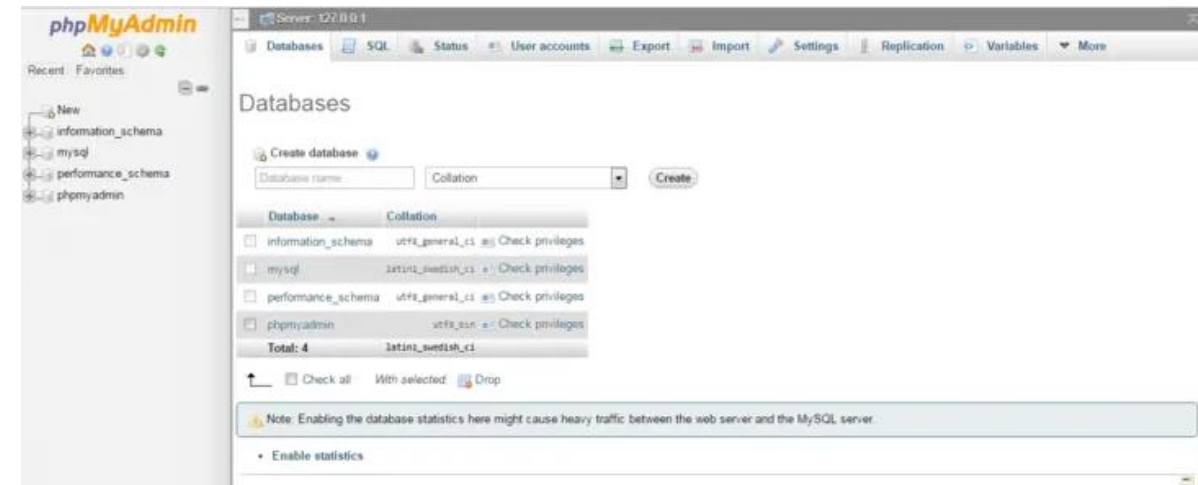
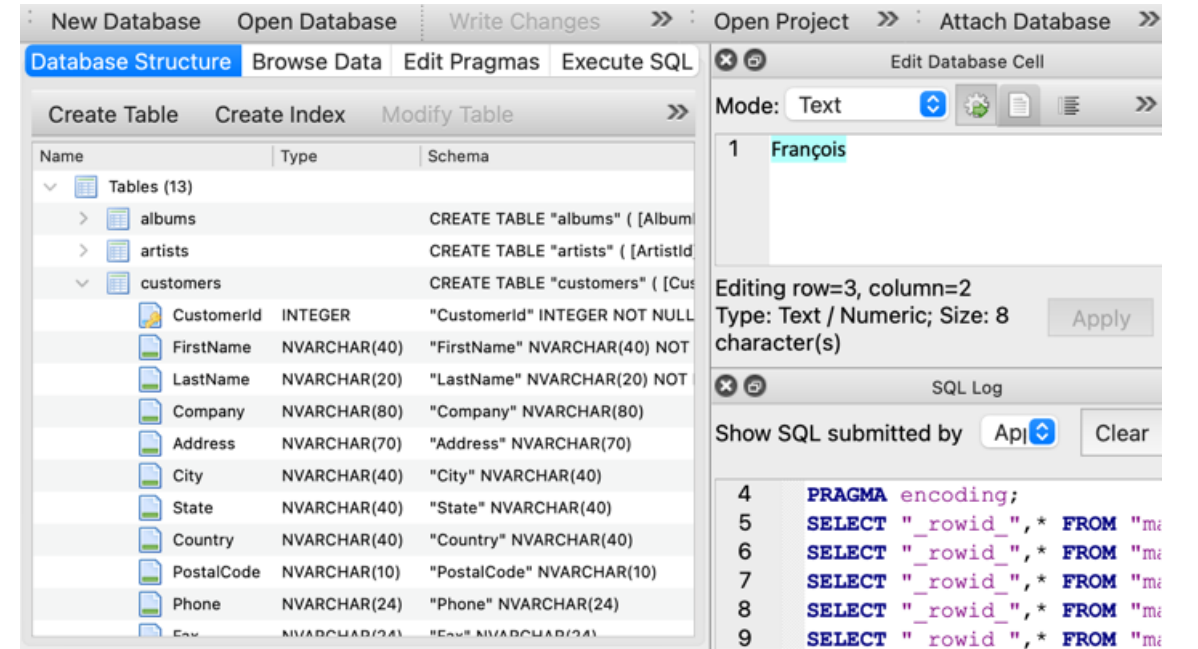
Advantages of database



A Database Management System

- A Database Management System (DBMS) is the software that **implements** a database

- Oracle
- DB2
- Ingres
- PostgreSQL
- Microsoft SQL Server (MS Access)
- SQLite
- MySQL



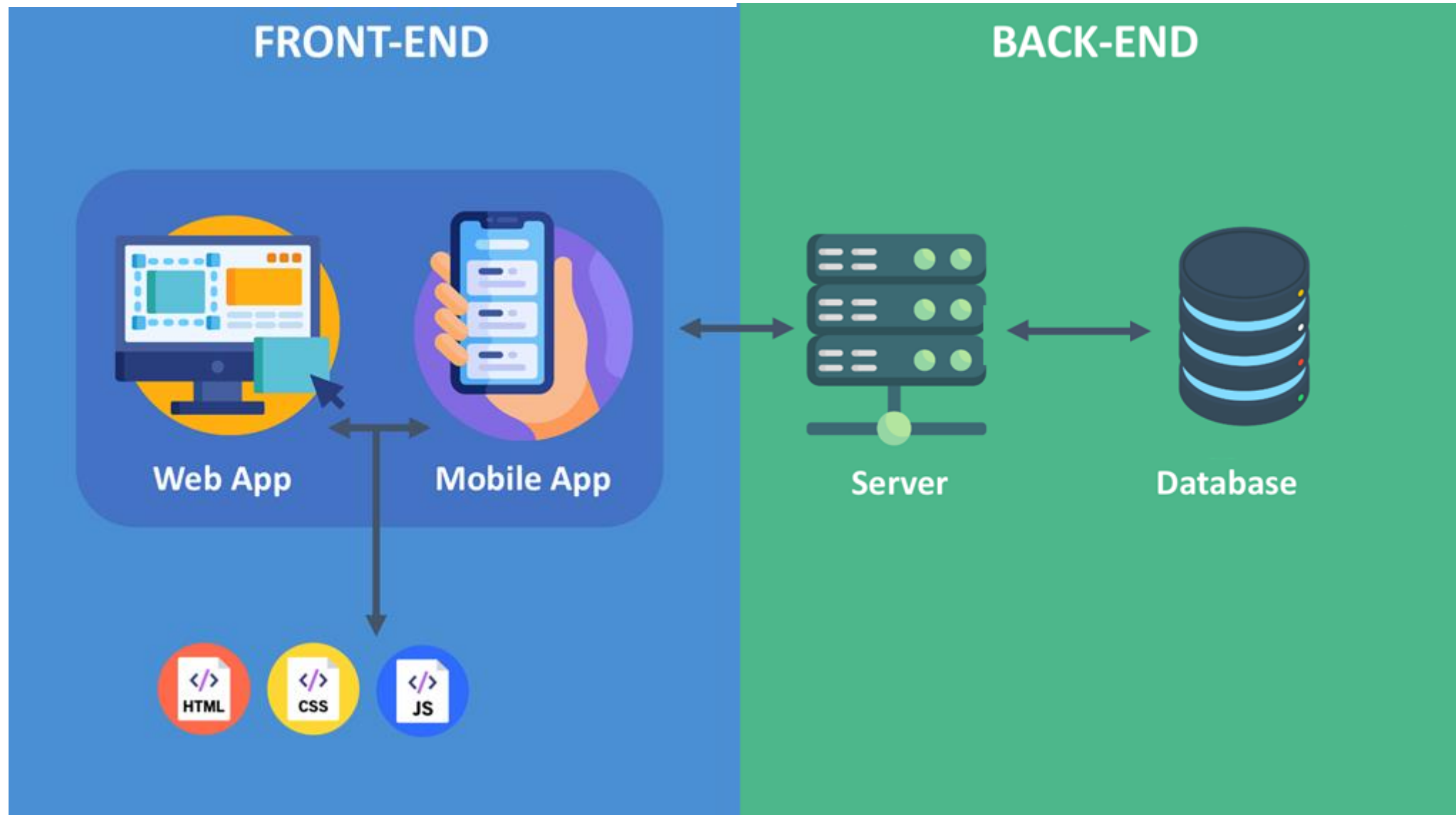
What a DBMS does

- Allows users to:
 - store data
 - manage change (updates)
 - organise data
 - retrieve data
 - retain privacy (security)
 - ensure reliability & consistency (no redundancy, etc.)

What a DBMS provides

- Data Definition Language (DDL)
 - e.g., 'CREATE' table (SQL), 'ALTER' table (SQL), 'DROP' table (SQL)
- Data Manipulation Language (DML)
 - e.g., 'INSERT' row (SQL), select row(s)
- Data Control Language (DCL)
 - e.g., 'GRANT' privilege on table to user (SQL)
- *These may be the same language: SQL=(DDL, DML, DCL)*

Where is database?

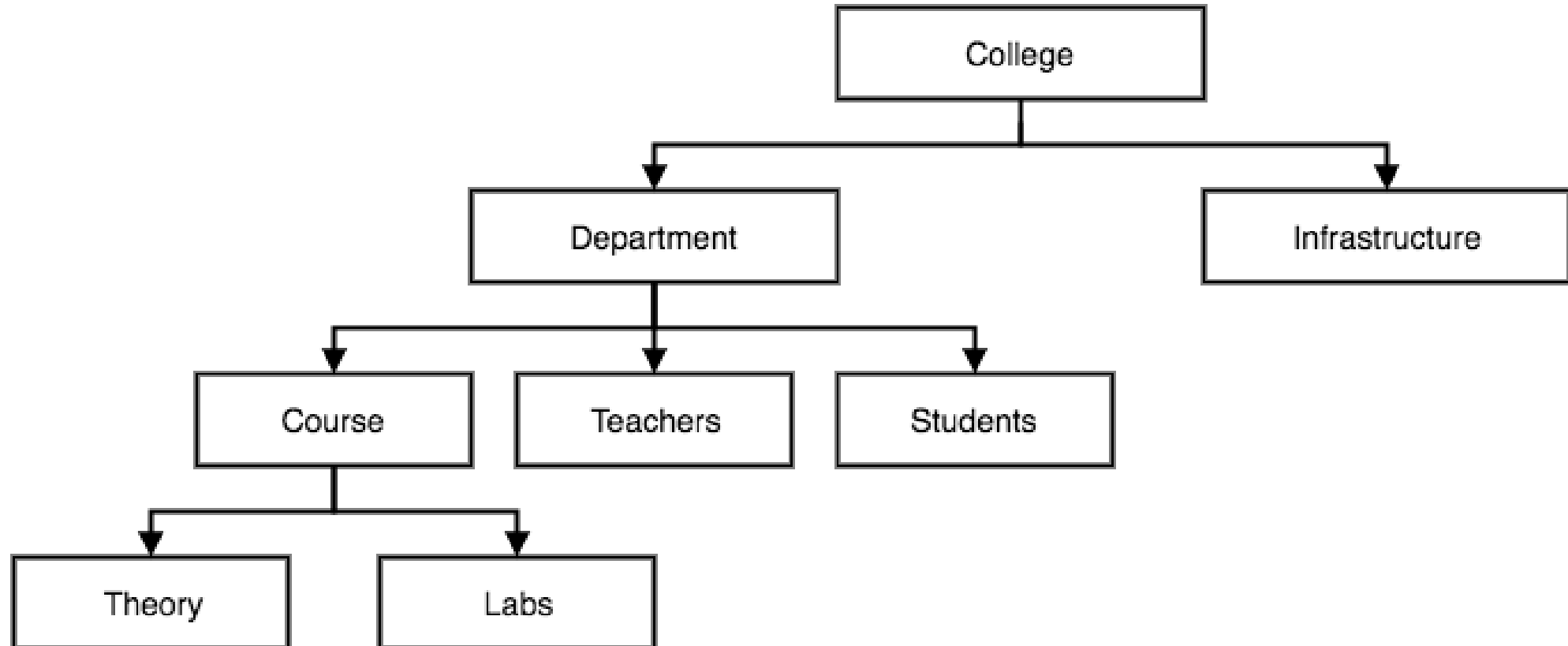


Types of database

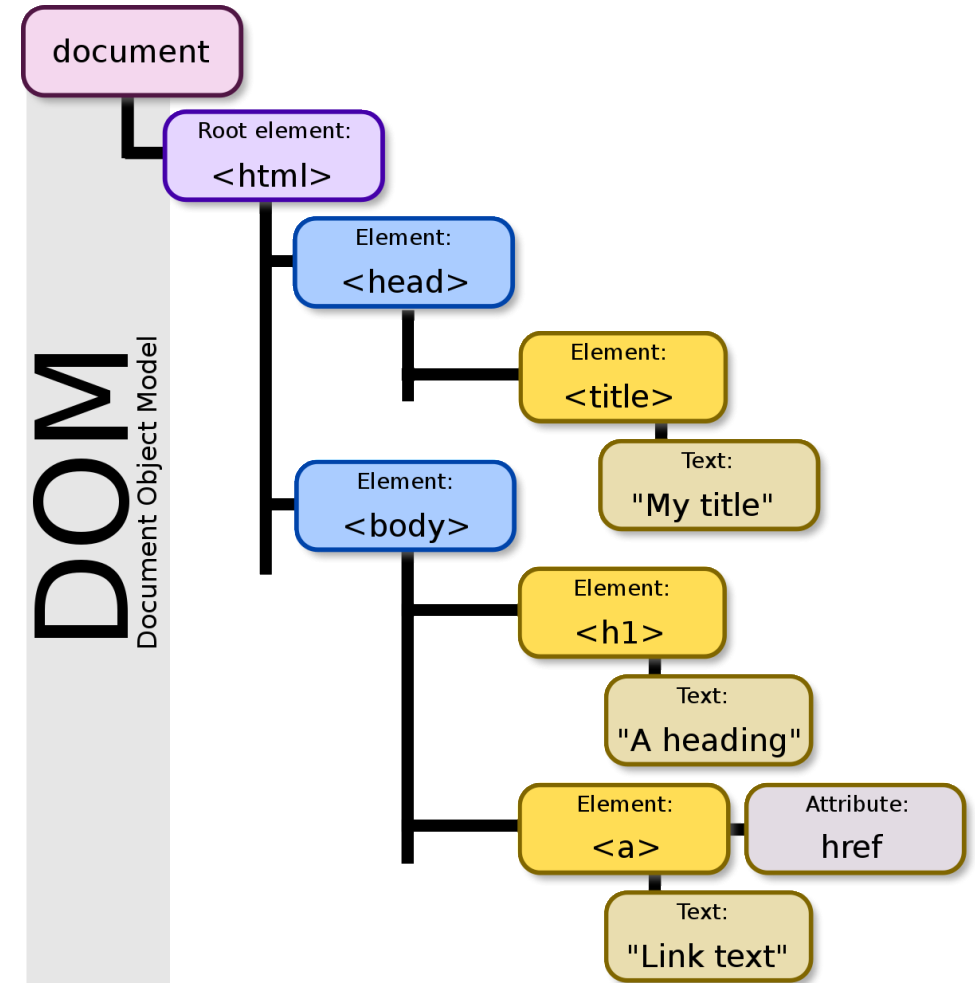
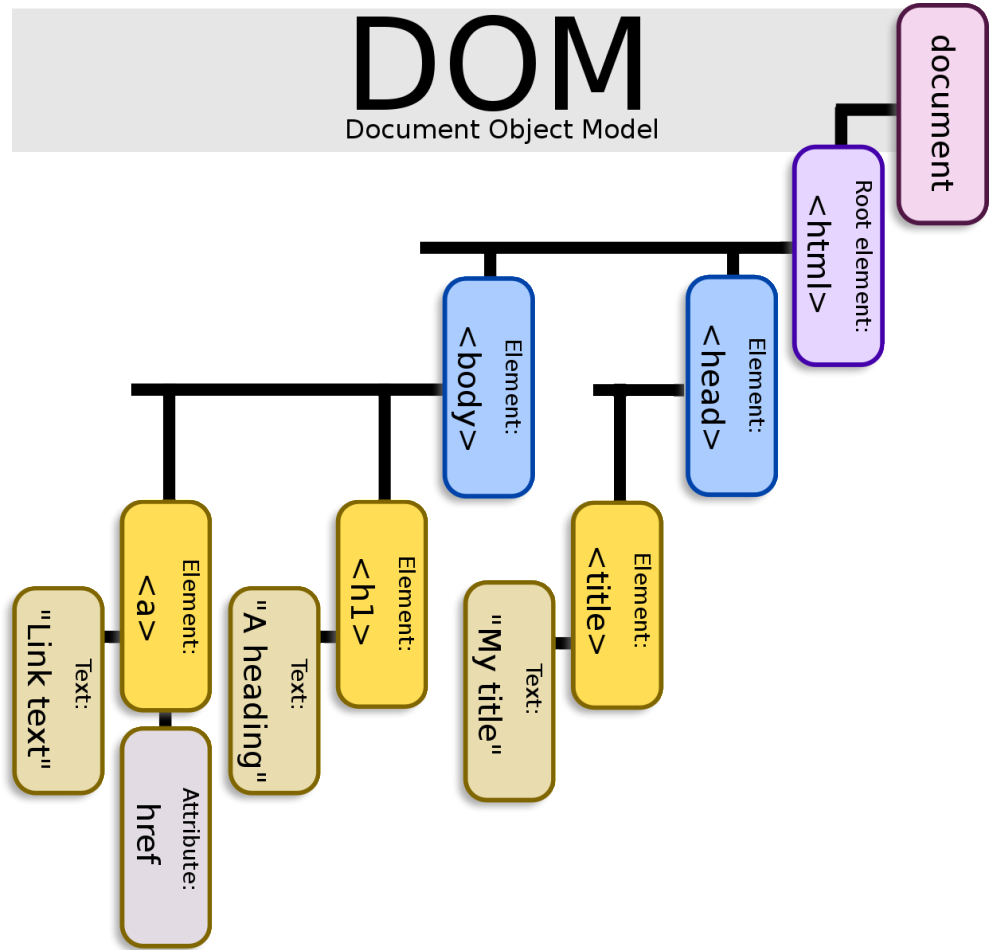
- There are several types of database (and DBMS)
- The main difference is their Data Model
 - the rules for the organisation of data (rows, columns etc.)
 - the mechanisms of association between data items (e.g., keys for relational databases)
 - the means by which data may be manipulated (e.g., operators)
- Examples of data model:
 - hierarchical data model
 - network data model
 - relational data model

Hierarchical data model

- Data is organised into a tree structure
 - records are generally linked to only one parent record
 - management requires knowledge of physical location and order of data



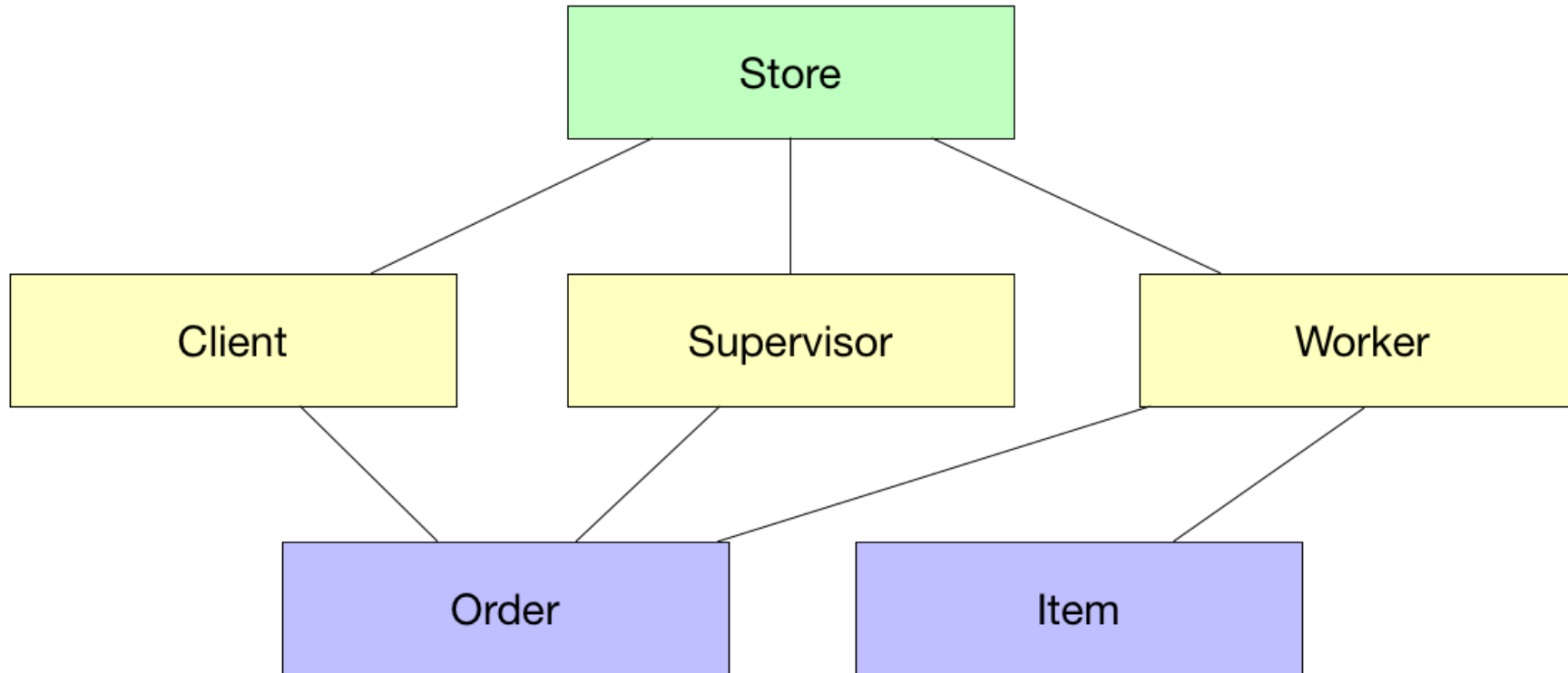
Hierarchical example: The DOM

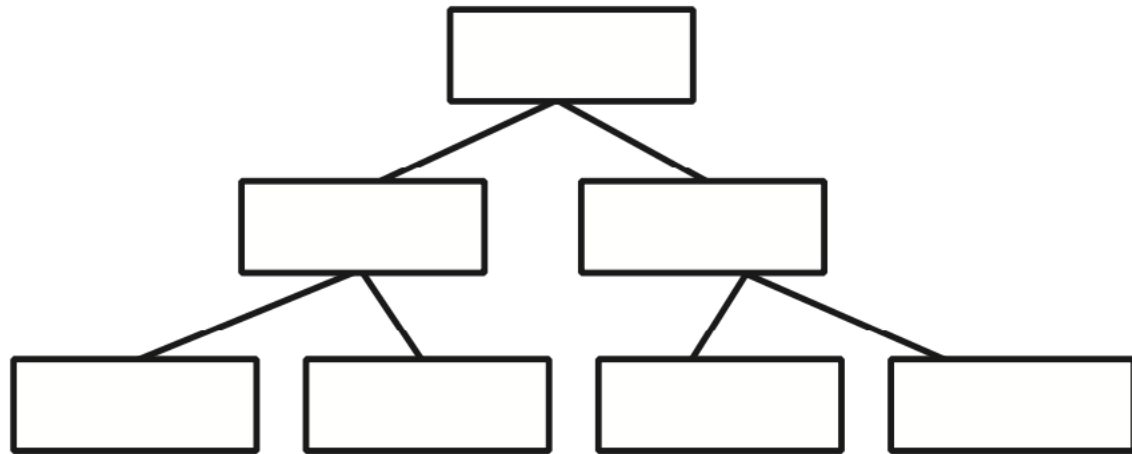


https://en.wikipedia.org/wiki/Document_Object_Model

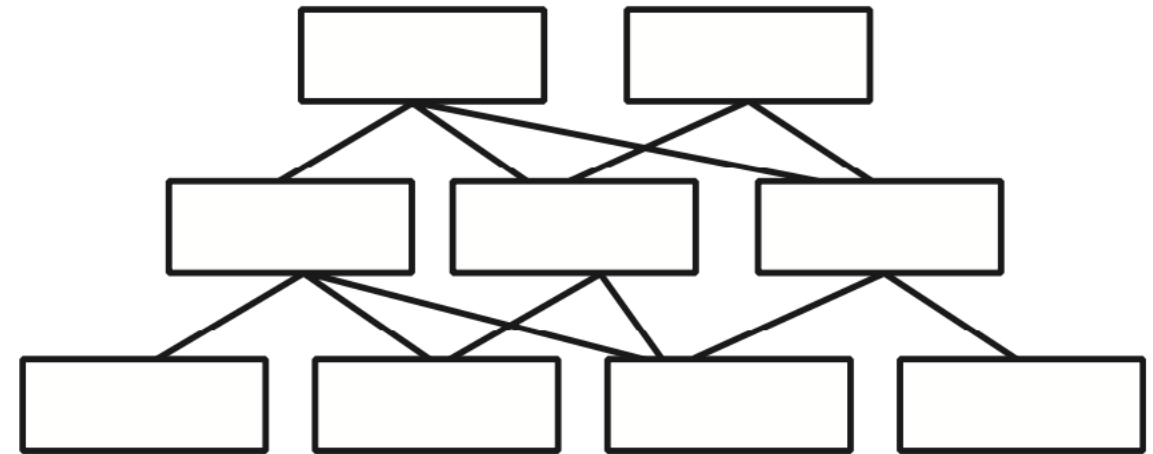
Network data model

- Data is organised into a loose network structure
 - records may be linked arbitrarily (even circularly)
 - management still requires knowledge of physical location and order of data





Hierarchical data model



Network data model

Relational data model

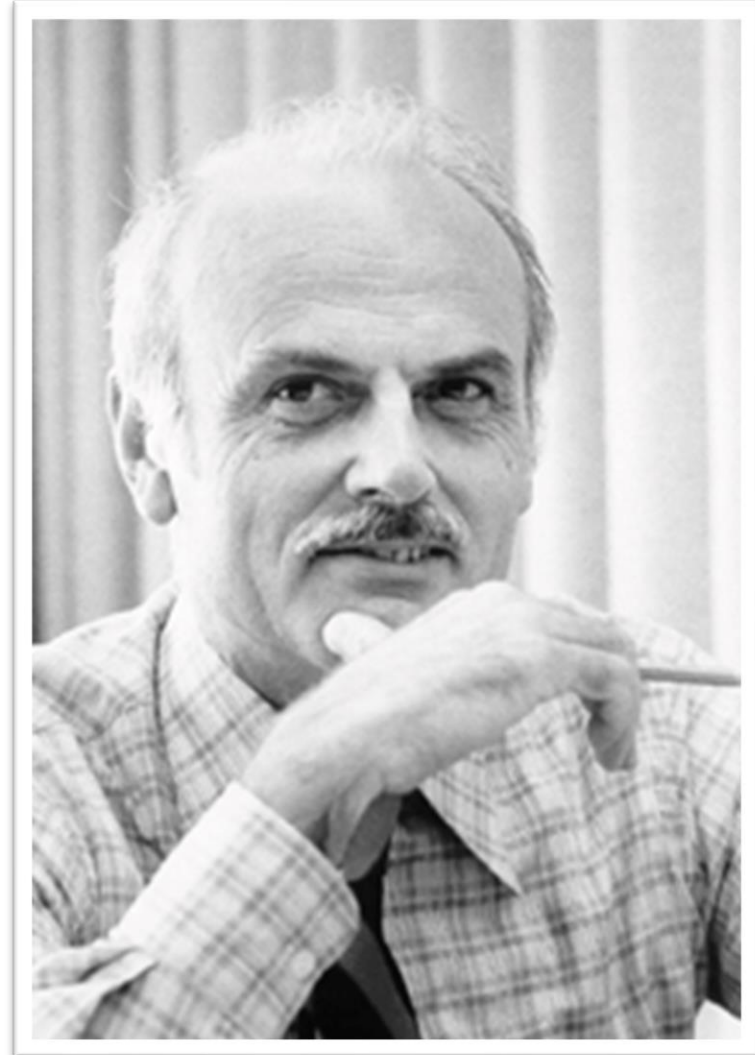
- Data is organised into **sets** ('statements of fact') – a key difference
 - sets are usually visualised as a table – for example:
 - the set of product codes, names, prices
 - the set of data describing product 101 (rows)
- The foundation of most (but not all) current database systems

PRODUCT CODE	PRODUCT NAME	UNIT PRICE
101	MELON	800G
102	STRAWBERRY	150G
103	APPLE	120G
104	LEMON	200G

Edgar Codd (1923 – 2003)

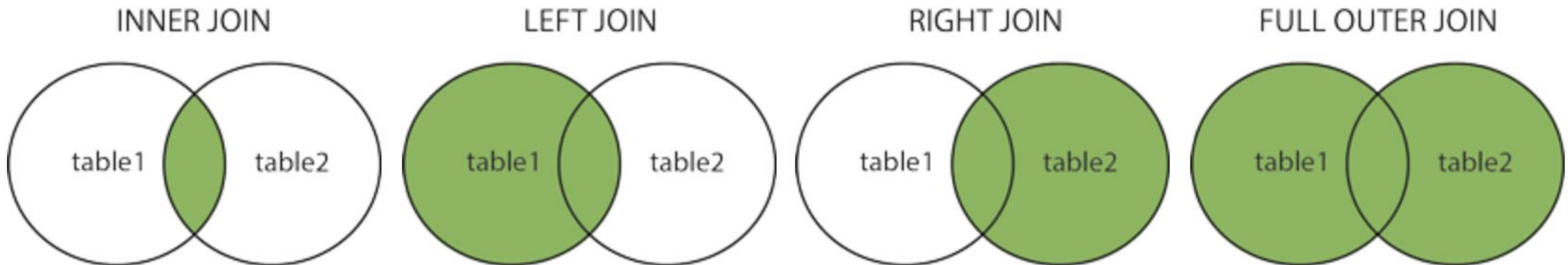
- British mathematician and theoretical computer scientist
- Worked on data management in IBM in the 1960s & 70s
- Invented the relational data model

Codd, E.F. (1970). A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM* 13 (6) 377–387.



Features of the relational model

- The Relational Data Model is concerned with:
 - Data structure (how data is represented)
 - Data integrity (what data is allowed)
 - Data manipulation (what you can do with the data)
- Important aspects of the relational model:
 - reducing data redundancy (normalisation)
 - set-like operations e.g., JOIN



The relational model

Simple database terminology

- The main unit of data in a database is a **Record** (row)
- Items of data in a record are **Fields** (column)
- Each record contains fields of the *same type*
- Records and Fields are organised into a **Table**

RECORD	REPORT CODE	DATE	EXPORT DEST. CODE	EXPORT DEST. NAME	PRODUCT CODE	PRODUCT NAME	UNIT PRICE	QUANTITY
	1101	3/5	12	THE KINGDOM OF MINANMI	101	MELON	800G	1,100
					102	STRAWBERRY	150G	300
	1102	3/7	23	ALPHA EMPIRE	103	APPLE	120G	1,700
	1103	3/8	25	THE KINGDOM OF RITOL	104	LEMON	200G	500
	1104	3/10	12	THE KINGDOM OF MINANMI	101	MELON	800G	2,500
	1105	3/12	25	THE KINGDOM OF RITOL	103	APPLE	120G	2,000
					104	LEMON	200G	700
FIELD (integer)								

COMP10

Relational data structure

- Data is stored in RELATIONS (tables)
- Each relation has a SCHEME (heading)
- A scheme defines the relation's ATTRIBUTES (concretely, results in columns containing fields)
- Data: takes the form of TUPLES (rows / records, made of fields)

	ATTRIBUTE	ATTRIBUTE	ATTRIBUTE
SCHEME	PRODUCT CODE	PRODUCT NAME	UNIT PRICE
TUPLE	101	MELON	800G
TUPLE	102	STRAWBERRY	150G
TUPLE	103	APPLE	120G
TUPLE	104	LEMON	200G

Attributes and domains

- A **domain** is given for each attribute
- The domain lists the possible values for that attribute
- Each tuple (row) contains a value for each attribute *from the domain* of the attribute (a collection of fields of specified types)
- Examples:
 - An 'age': member of the set of integers between 0 and 150
 - A 'date & time': specific representational form e.g., 12:30:00 05/10/2020, etc.
 - A 'product name': from a given set of specified strings
 - Perhaps itself another relation?
 - A 'notes' field might allow any arbitrary string at all

Key Definitions

- Tuple
 - A tuple is a row of a relation
- Degree
 - The degree of a relation is the number of attributes it contains
- Cardinality
 - The cardinality of a relation is the number of tuples it contains
- Relational database (Relational database schema)
 - A collection of normalised relations with distinct relation names

A relation (table)

3 Attributes (Degree = 3)

Heading /
scheme

4 Tuples

(Cardinality = 4)

product_id	product_name	product_unit_price
101	MELON	800G
102	STRAWBERRY	150G
103	APPLE	120G
104	LEMON	200G

Relation

Properties of Relations

- the relation has a name that is distinct from all other relation names in the relational schema
- each cell of the relation contains exactly one atomic (single) value
- each attribute has a distinct name
- the values of an attribute are all from the same domain
- each tuple is distinct; there are no duplicate tuples
- the order of attributes has no significance

Keys

- CANDIDATE KEYs are unique fields
- One candidate key is *usually* declared to be the PRIMARY KEY
 - PRIMARY KEY is a field that serves to identify data
- A FOREIGN KEY is a key in another table

PRODUCT CODE	PRODUCT NAME	UNIT PRICE
101	MELON	800G
102	STRAWBERRY	150G
103	APPLE	120G
104	LEMON	200G

PRIMARY KEY

Candidate keys

- Set of attributes in a relation is called a **Candidate Key** if it meets the following criteria:
 - set of attributes uniquely identify a tuple (*uniqueness*)
 - we cannot get rid of any value without losing uniqueness (*minimality*)
- Candidate keys are:
 - {ID} and {First, Last}
- Not unique:
 - {First} and {Last}
- Not minimal:
 - {ID, First} {ID, Last} and {ID, First, Last}

ID	FIRST	LAST
S139	John	Smith
S140	Mary	Jones
S141	John	Brown
S142	Jane	Smith

Primary keys

- One Candidate Key is *usually* chosen to be used to identify tuples in a relation (but not necessarily) – the **Primary Key**
- *Usually* a special ID attribute is used as the primary key
- Either {ID} or {First, Last} could be used as the primary key
- ID is a ‘better’ primary key since it is a single column, and it will always be unique

ID	FIRST	LAST
S139	John	Smith
S140	Mary	Jones
S141	John	Brown
S142	Jane	Smith

NULLs and primary keys

- Missing information can be represented using **NULL**
- NULL indicates an unknown or missing value
- Primary keys cannot contain NULL values
 - (AKA entity integrity)
- NOTE: NULLs can cause problems
 - we have no way of knowing whether a value is unknown or missing
 - NULLs are intrinsically uncertain
 - uncertainty is difficult to deal with computationally

ID	FIRST	MIDDLE	LAST
S139	John	Arthur	Smith
S140	Mary	NULL	Jones
S141	John	NULL	Brown
S142	Jane	Anne	Smith

Foreign keys

- Foreign Keys are used to link data in two relations.
- A set of attributes in the child (**Referencing**) relation is a Foreign Key if its value always matches a Candidate Key value in the parent (**Referenced**) relation

Foreign keys: example

Department	
dept_id	dept_name
13	Marketing
14	Accounts
15	Personnel

dept_id is a **candidate key** for **department**

Employee		
empl_id	empl_name	dept_id
15	John Smith	13
16	Mary Brown	14
17	Mark Jones	13
18	Jane Smith	NULL

dept_id is a **foreign key** for **employee**. Each dept_id value is either NULL or matches an entry in the **department** relation

Relational data integrity

- Data integrity controls what data can be in a relation
- This relates to:
 - **Domains** restrict the possible values of attributes
 - **Primary Key** identify tuples within a relation
 - **Foreign Keys** link relations to each other

Integrity Constraints

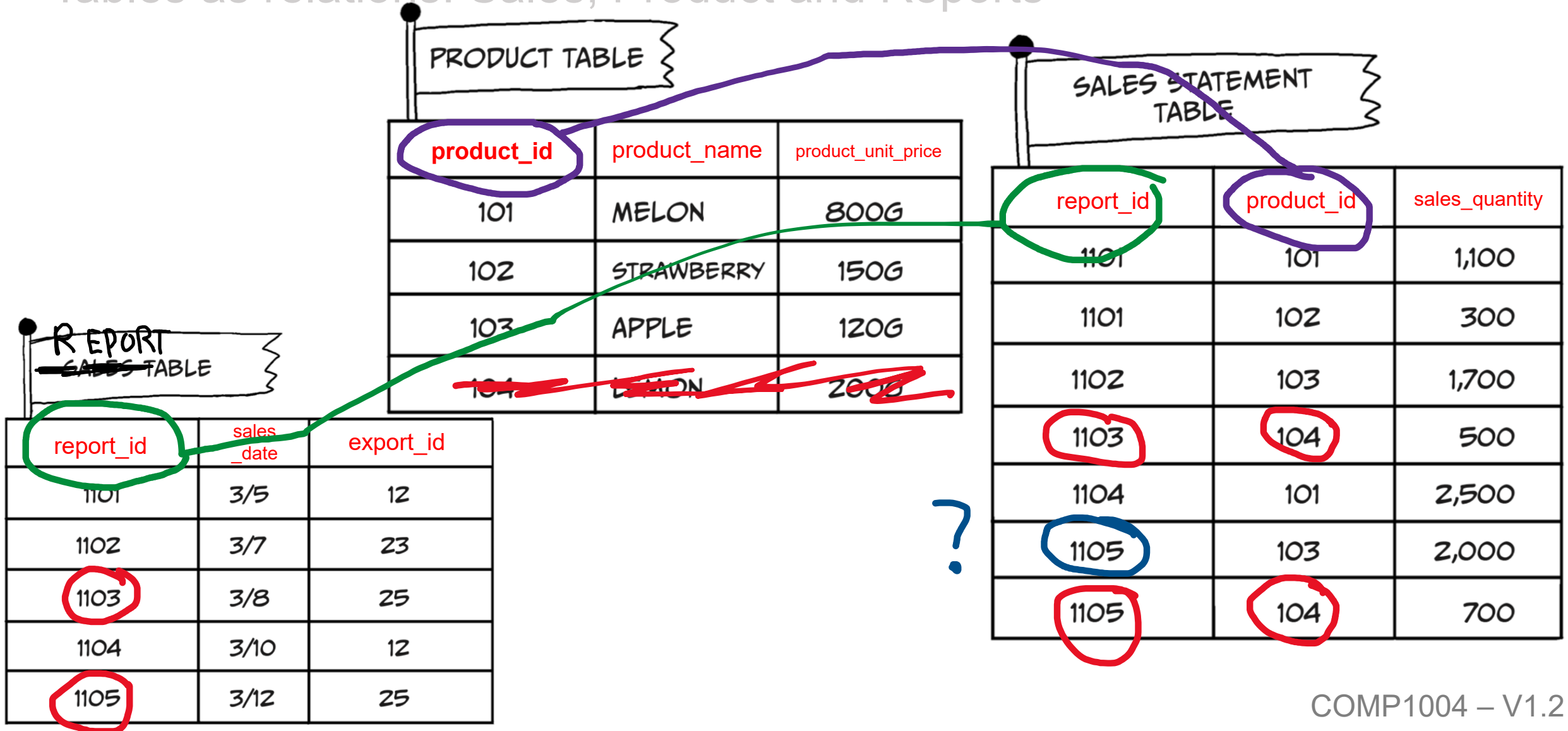
- Null
 - Represents a value for an attribute that is currently unknown or is not applicable for this tuple
- Entity integrity
 - In a base relation, primary key can not be null
- Referential integrity
 - If a foreign key exists in a relation, either the foreign key value must match a candidate key value of some tuple in its home relation or the foreign key value must be wholly null

Dealing with referential integrity

- When relations are updated, referential integrity can be violated
- This usually occurs when a referenced tuple (record / row) is updated or deleted

Example (*MGDB*, p. 70)

- Tables as relations: Sales, Product and Reports



Dealing with referential integrity

- When relations are updated, referential integrity can be violated
- This usually occurs when a referenced tuple (record / row) is updated or deleted
- There are a number of options:
 - **RESTRICT**: stop the user from doing it
 - **CASCADE**: let the changes flow on
 - **NULLIFY**: make values NULL
 - Triggers – actions defined by the user

Referential integrity: example

- What happens if:
 - Marketing's dept_id is changed to 16 in department?
 - The entry for Accounts is deleted from Department?

department	
dept_id	dept_name
14 16	Marketing
14	Accounts
15	Personnel

employee		
empl_id	empl_name	dept_id
15	John Smith	13
16	Mary Brown	14
17	Mark Jones	13
18	Jane Smith	NULL

Restrict

- RESTRICT stops any action that violates integrity
 - You cannot update or delete Marketing or Accounts
 - You can change Personnel as it is not referenced

department	
dept_id	dept_name
13	Marketing
14	Accounts
15	Personnel

employee		
empl_id	empl_name	dept_id
15	John Smith	13
16	Mary Brown	14
17	Mark Jones	13
18	Jane Smith	NULL

Cascade

- CASCADE allows the changes made to flow through
 - If Marketing's dept_id is changed to 16 in department, then the dept_ids for John Smith and Mark Jones also change
 - If Accounts is deleted then so is Mary Brown

department	
dept_id	dept_name
13 16	Marketing
14	Accounts
15	Personnel

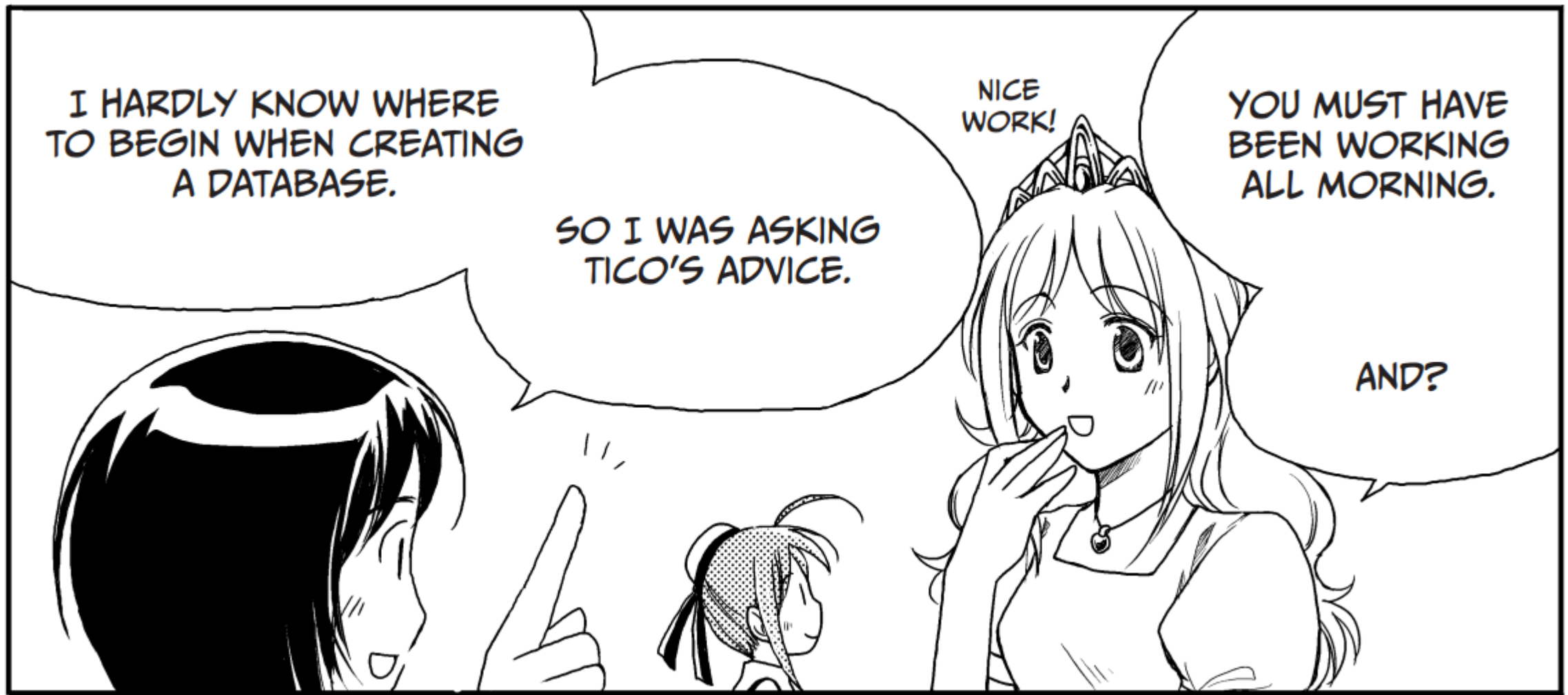
employee		
empl_id	empl_name	dept_id
15	John Smith	13 16
16	Mary Brown	14
17	Mark Jones	13 16
18	Jane Smith	NULL

Nullify

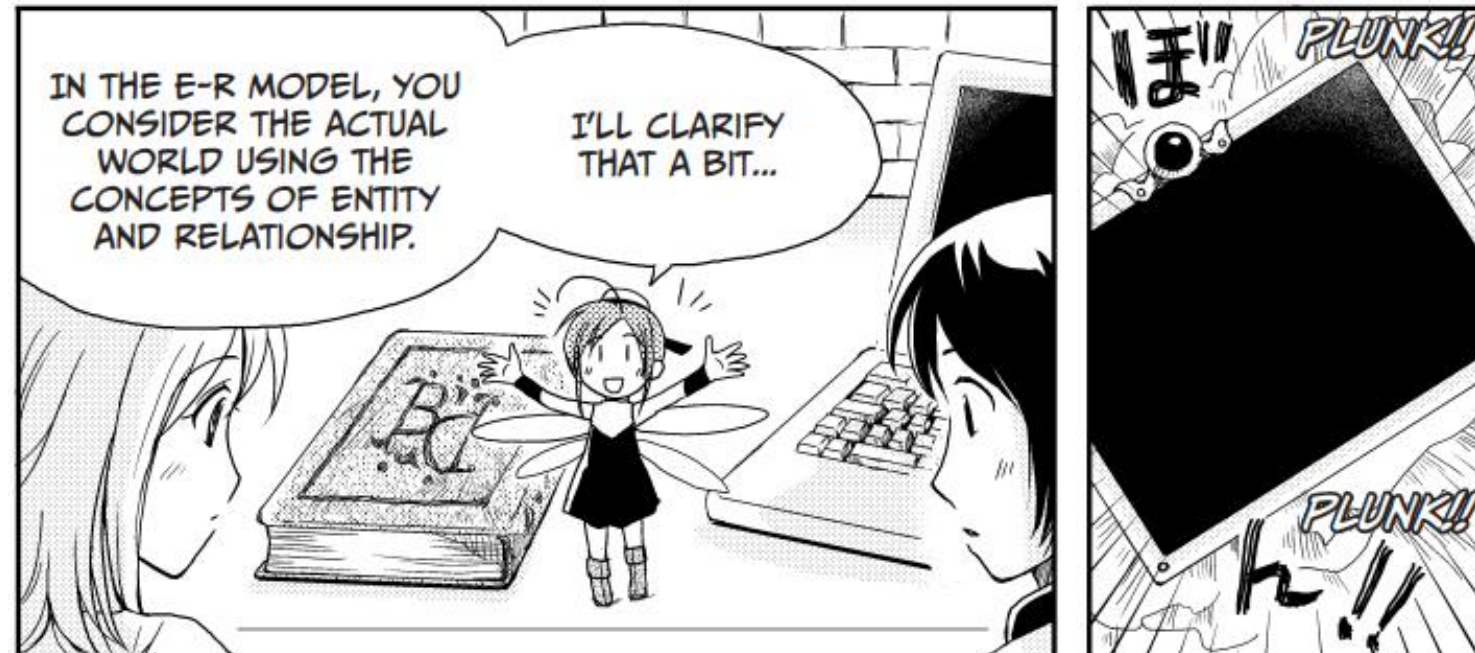
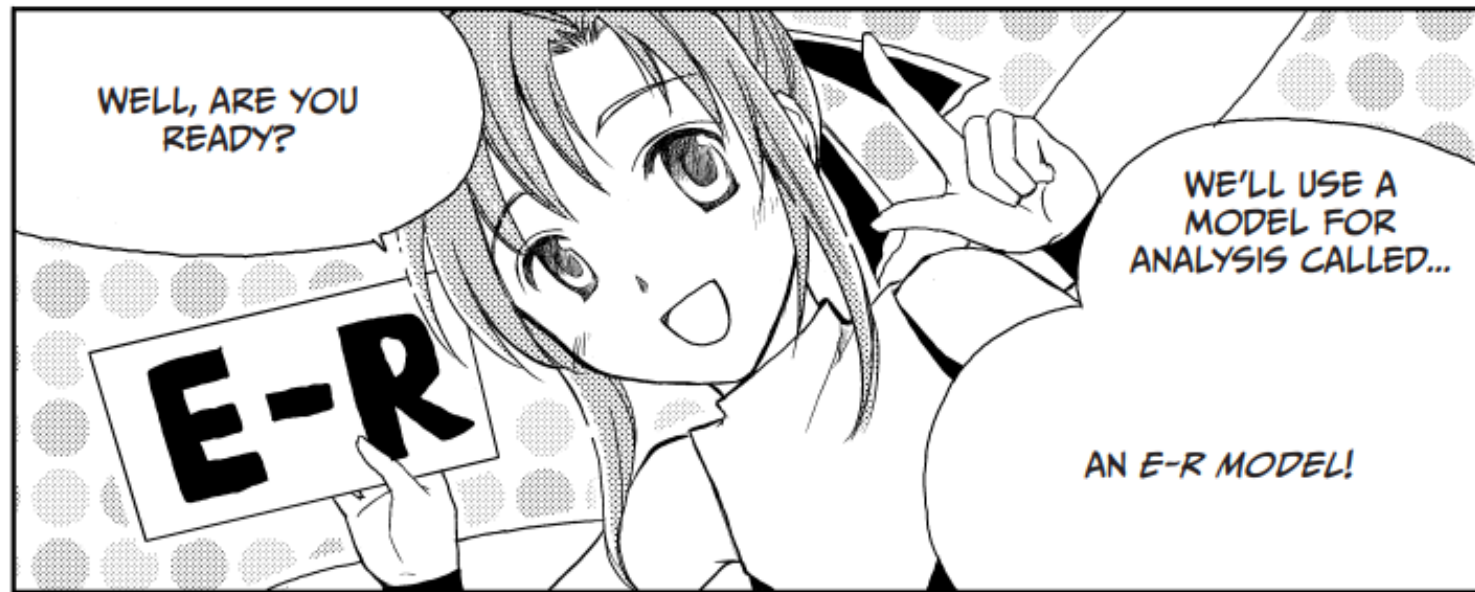
- NULLIFY sets problem values to NULL
 - If Marketing's dept_id changes then John Smith's and Mark Jones' dept_ids are set to NULL
 - If Accounts is deleted, Mary Brown's dept_id becomes NULL

department	
dept_id	dept_name
13 16	Marketing
14	Accounts
15	Personnel

employee		
empl_id	empl_name	dept_id
15	John Smith	13 NULL
16	Mary Brown	14 NULL
17	Mark Jones	13 NULL
18	Jane Smith	NULL

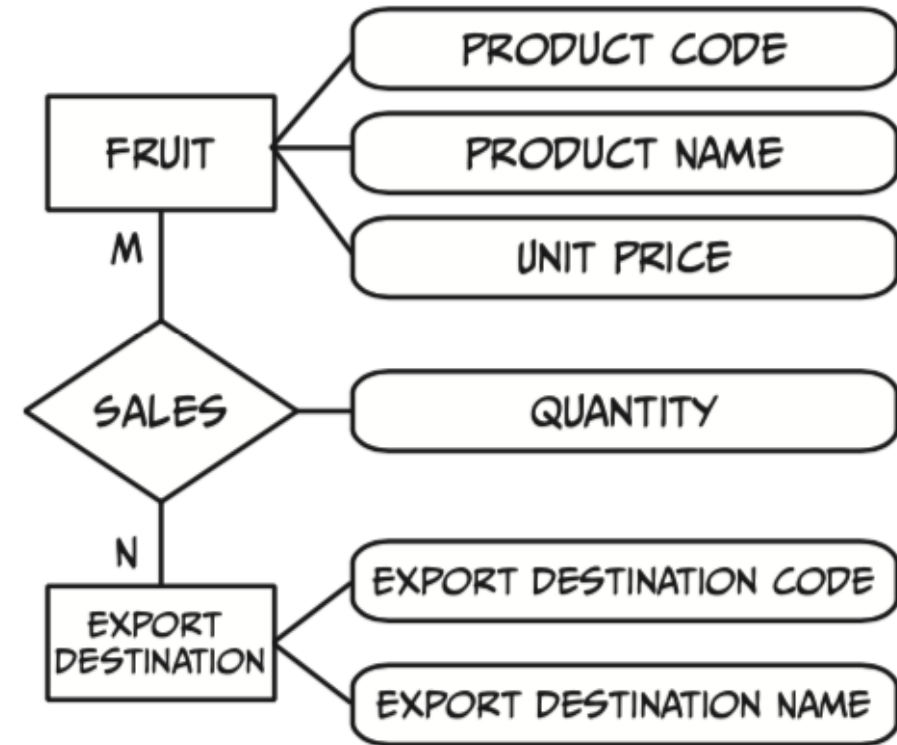


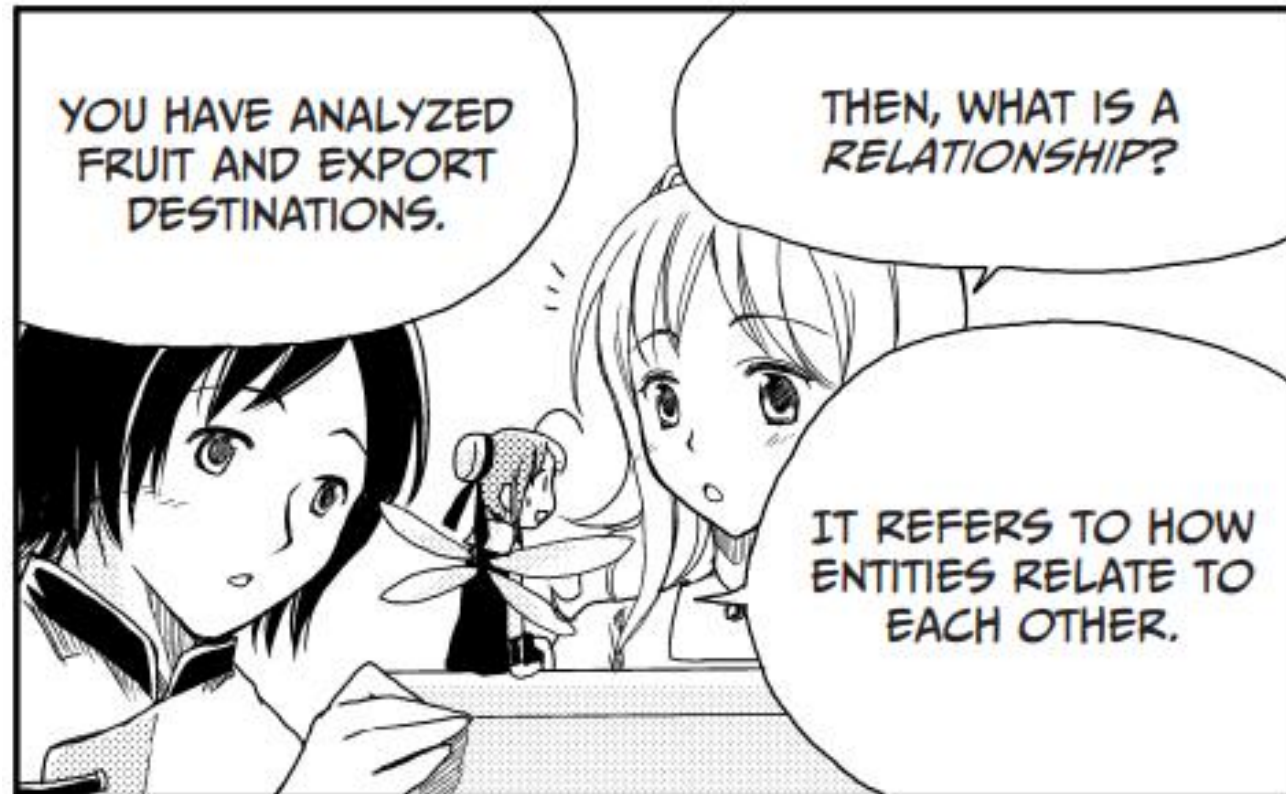
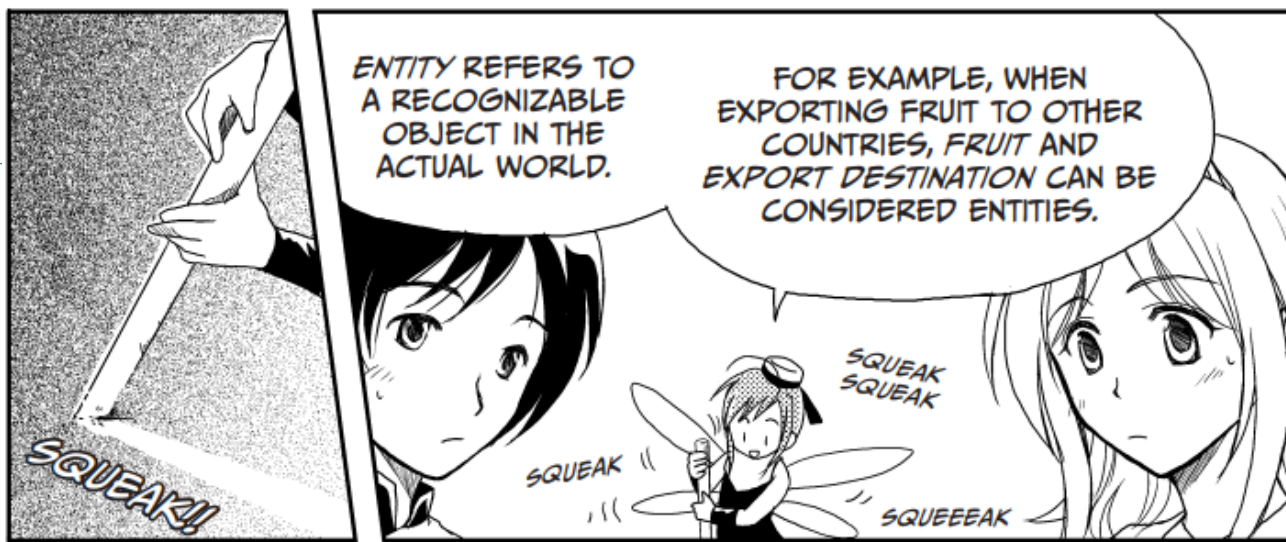
Entity Relationship Modelling



Entity-Relationship Modelling

- Aim: Create a **visual model** of the database to discuss with the client
 - E.g., Entity-Relationship diagram
- Next stage in the design of the database
- ER diagrams capture the design of the database
 - Entities (tables & fields)
 - Relationships between entities





Entity-Relationship Modelling

- Entity
Real world objects/items/concepts of interest
- Attribute
Properties (of interest) of entities
- Relationship
Links between entities
- E-R Modelling
Translating requirements into a E-R model

E-R Notation

- Various conventions
 - Often vary by country
- Most common ones:
 - Chen
 - Mostly American
 - Crow's Foot
 - Mostly British

Entities

- Entity - distinguishable “thing” in the real world
 - **Strong (or regular) entity** - entities have an independent existence (e.g. staff)
 - **Weak entity** - existence dependent on some other entity (e.g. next of kin)

ER COMPONENT	CROWFOOT NOTATION	CHEN NOTATION
Entity		
Weak Entity		

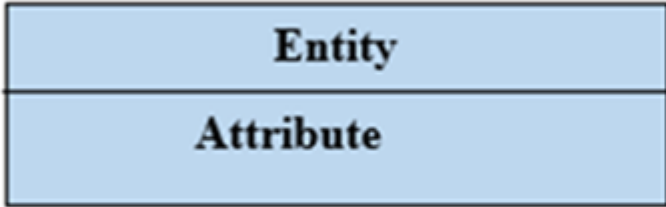
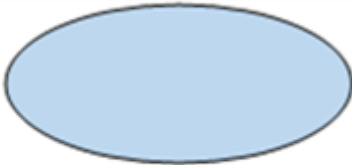
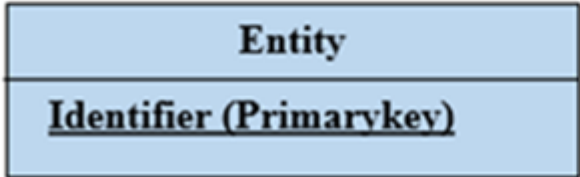
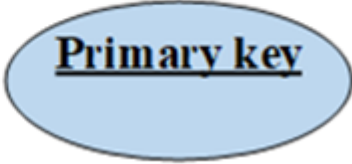
Scenario

- An online shopping system records data about customers, orders, and products. A customer can place multiple orders, but each order is placed by exactly one customer. An order can contain multiple products, and the same product can appear in many orders.
- Entities???



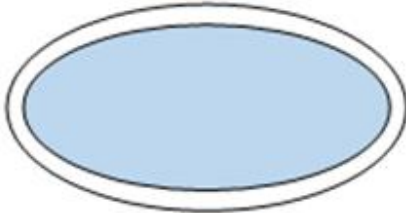
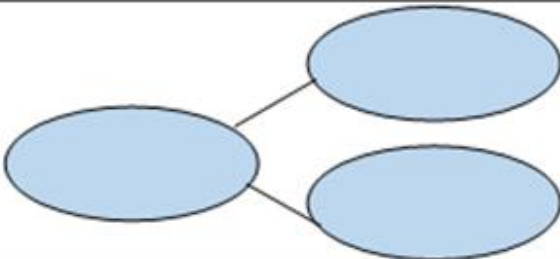
Attributes(1)

- Entity have Attributes (or properties) which associate each entity with a value from a domain of values for that attribute
- Attributes can be
- Simple (or atomic) attribute
 - Cannot be broken down into smaller components (color,age)

ER COMPONENT	CROWFOOT NOTATION	CHEN NOTATION
Attribute		
Key attribute (primary key)		

Attributes(2)

- Composite attribute
 - Can be broken down into smaller components
 - Example: Address: House number, Street, City, State, and Postal Code
- Multivalued attribute
 - May take on more than one value
 - Example: Skill, Hobby

ER COMPONENT	CROWFOOT NOTATION	CHEN NOTATION		
Multivalued attribute	<table><tr><td>Entity</td></tr><tr><td>{Multivalued}</td></tr></table>	Entity	{Multivalued}	
Entity				
{Multivalued}				
Composite attribute	<table><tr><td>Entity</td></tr><tr><td>Composite(,,)</td></tr></table>	Entity	Composite(,,)	
Entity				
Composite(,,)				

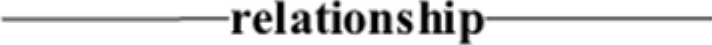
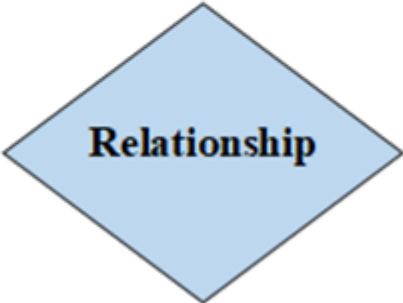
Scenario

- An online shopping system records data about customers, orders, and products. A customer can place multiple orders, but each order is placed by exactly one customer. An order can contain multiple products, and the same product can appear in many orders.
- Attributes???



Relationships

- A relationship is
 - “.. An association among entities (the participants)
- Relationships link entities with each other

ER COMPONENT	CROWFOOT NOTATION	CHEN NOTATION
Relationship		

Scenario

- An online shopping system records data about customers, orders, and products. A customer can place multiple orders, but each order is placed by exactly one customer. An order can contain multiple products, and the same product can appear in many orders.
- Relationship???

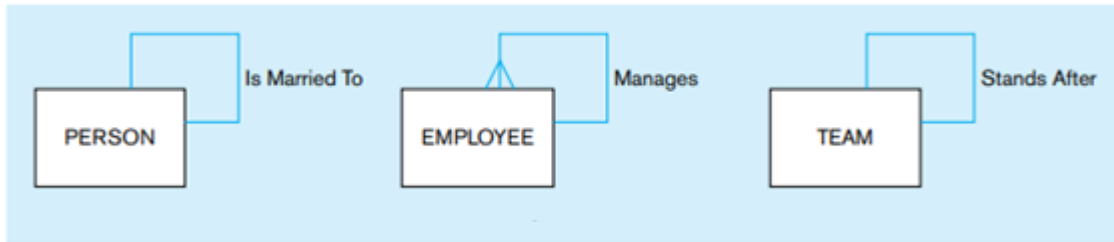


Relationships: Constraints

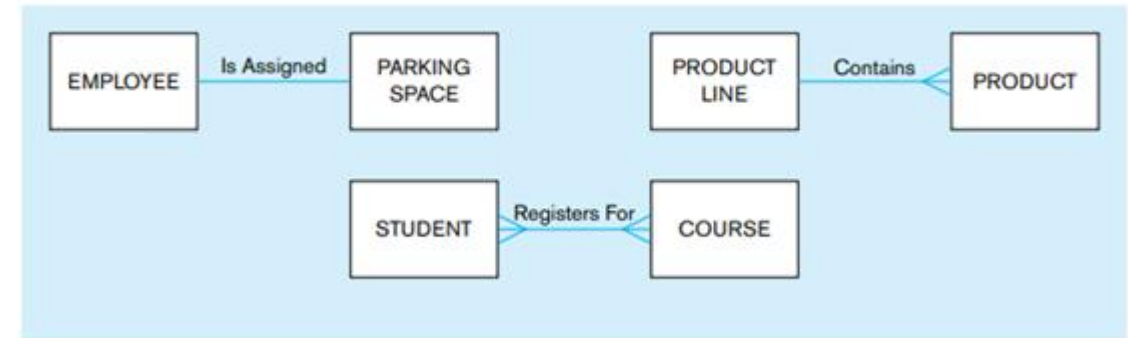
- The **degree** of a relationship
 - Unary/ recursive (connects 1 entity type with itself)
 - Binary (connects 2 entity types)
 - Complex (connects 3 or more entity types)
 - Ternary (connects 3)

Degree

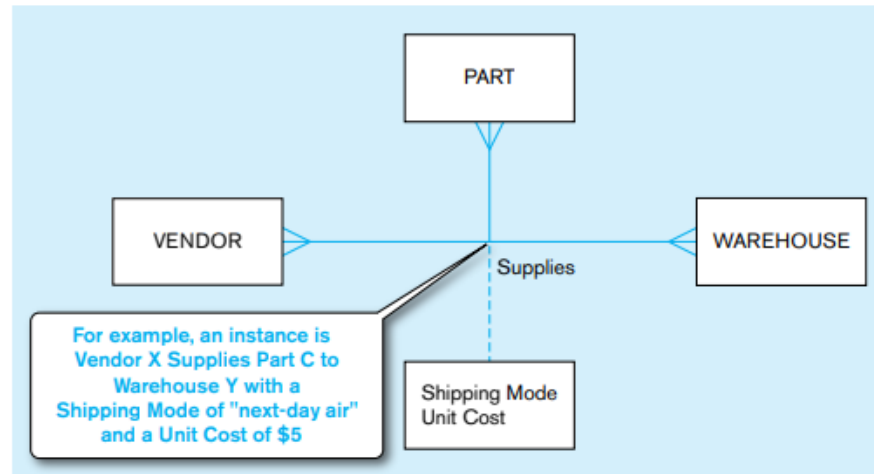
(a) Unary relationships



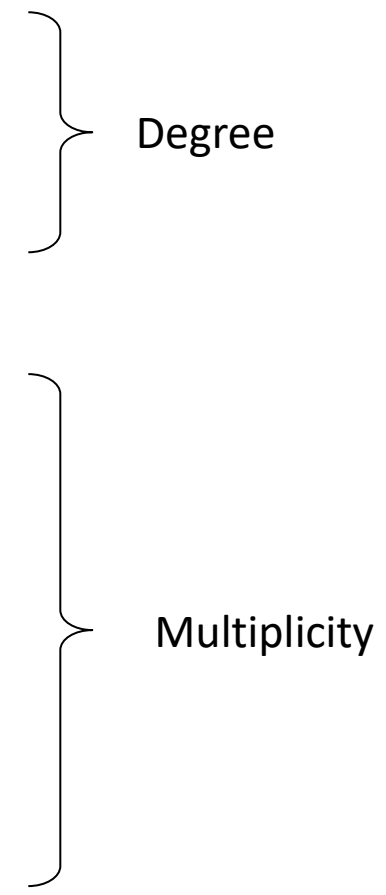
(b) Binary relationships



(c) Ternary relationship

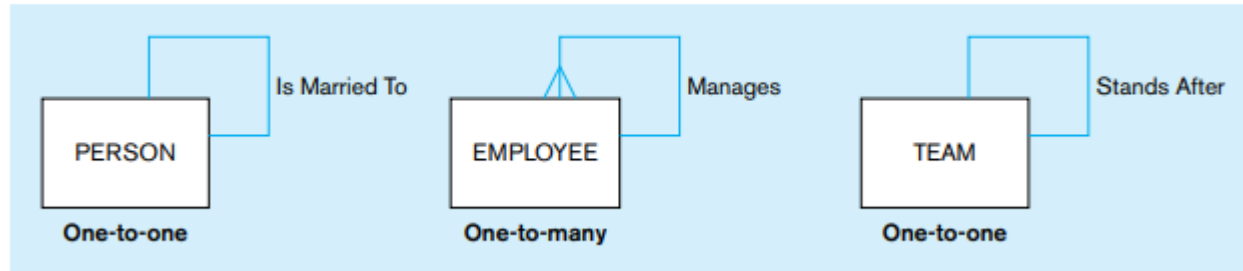


Relationships: Constraints

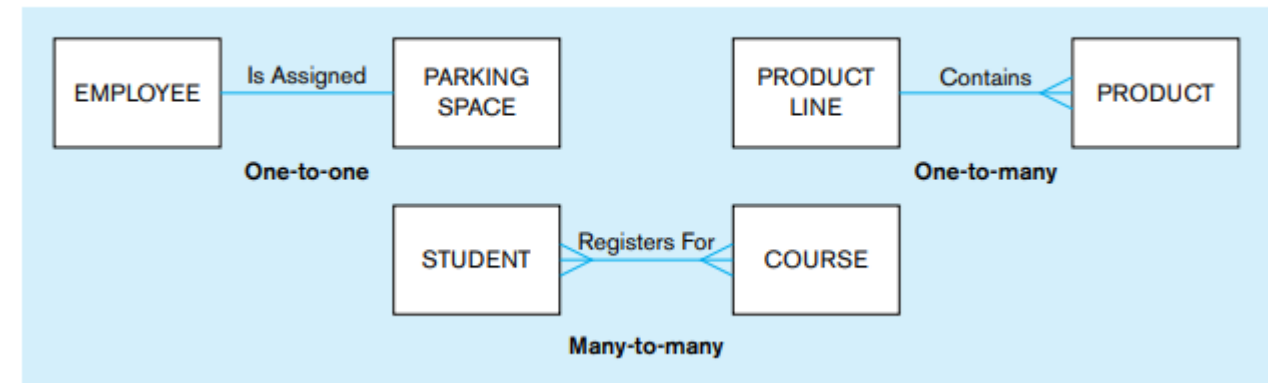
- The **degree** of a relationship type
 - Unary/ recursive (connects 1 entity type with itself)
 - Binary (connects 2 entity types)
 - Complex (connects 3 or more entity types)
 - Ternary (connects 3)
 - Relationship constraints - **cardinality**
 - One to one (one person is married to one person)
 - One to many (one person manage many person)
 - Many to many (student register module)
 - Relationship constraints – **participation**
 - Mandatory
 - Partial/optional
- 
- The diagram uses curly braces to group the list items into two categories. The first category, labeled 'Degree', groups the first three items: 'The degree of a relationship type' and its sub-items. The second category, labeled 'Multiplicity', groups the remaining three items: 'Relationship constraints - cardinality', 'Relationship constraints – participation', and its sub-items.

Relationships (Cardinality)

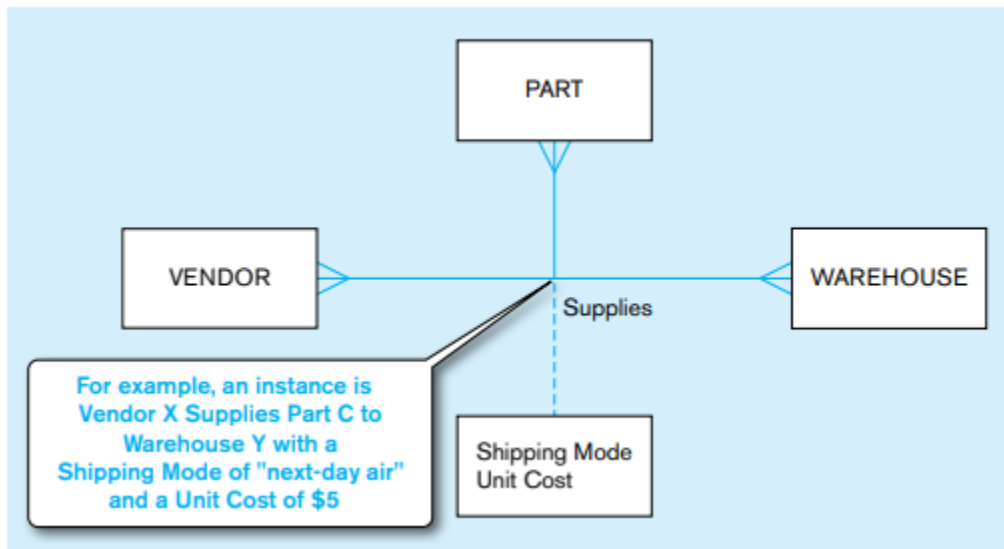
(a) Unary relationships



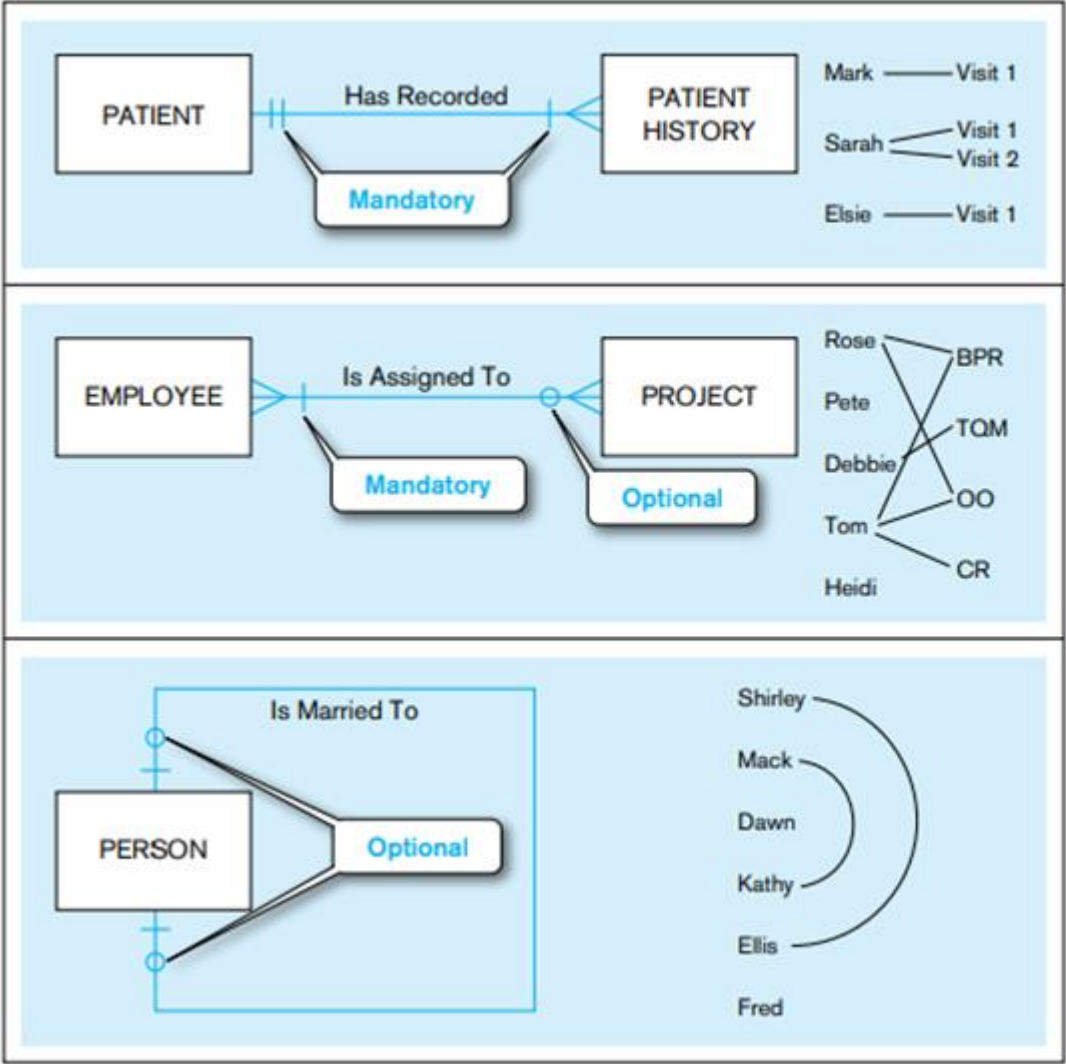
(b) Binary relationships



(c) Ternary relationship



Relationship (Participation)

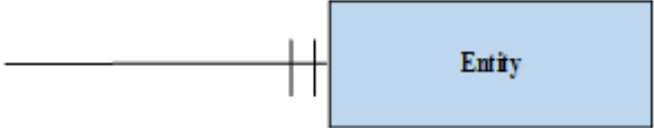
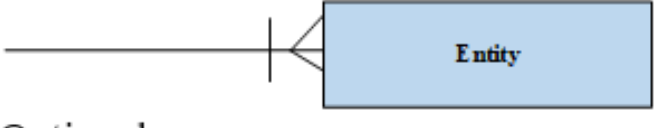
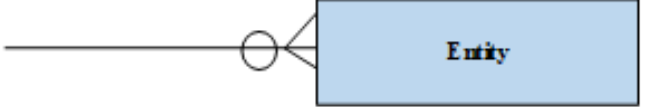


Examples of
cardinality constraints
(a) Mandatory cardinalities

(b) One optional, one mandatory
cardinality

(c) Optional cardinalities

E-R Notation

ER COMPONENT	CROWFOOT NOTATION	CHEN NOTATION
Relationship cardinality	Mandatory one 	Mandatory one <u>1..1</u>
	Mandatory Many 	Optional one <u>0..1</u>
	Optional one 	Optional Many <u>0..M</u>
	Optional Many 	Mandatory Many <u>1..M</u>
		In some books optional and mandatory are represented by dotted and non-dotted lines e.g. Mandatory one <u>1</u> Mandatory Many <u>M</u> Optional one <u>1</u> Optional Many <u>M</u>

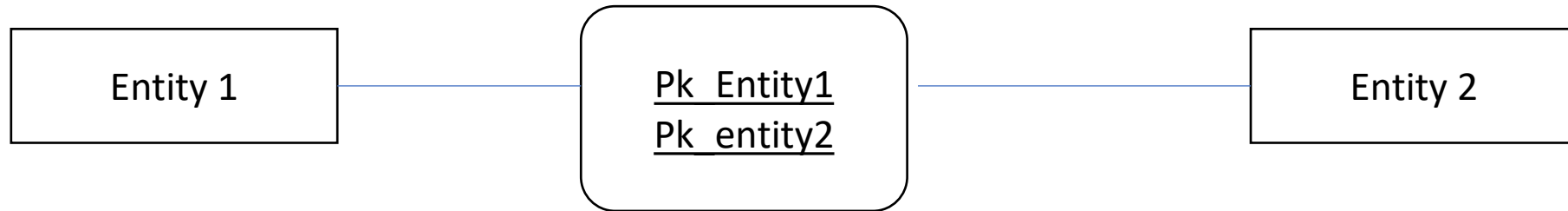
Scenario

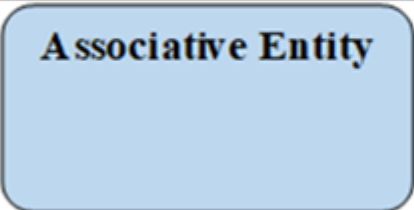
- An online shopping system records data about customers, orders, and products. A customer can place multiple orders, but each order is placed by exactly one customer. An order can contain multiple products, and the same product can appear in many orders.
- Cardinality???



Associative Entity

- Connects two entities that have multiple interactions with each other.
- Contains foreign keys that reference the primary keys of the two entities it links



ER COMPONENT	CROWFOOT NOTATION	CHEN NOTATION
Associative Entity		

ER modelling – example

Example 1

A university consists of a number of schools. Each school offers several courses. A particular course could only be offered by a single school. A number of modules make up each course where one single module could be in several courses. Students enrol in a particular course at a time and take modules towards the completion of that course. Each module is taught by a lecturer from the appropriate school, and each lecturer tutors a group of students.

Conceptual Design using E-R Models

- Given a description of an application, your task is to:
 - Identify entities
 - Since they are objects, typically described using nouns
 - Identify attributes
 - Facts or properties, so often also nouns
 - Identify relationships
 - Relationships are interactions between entities, so are usually described with verbs
 - Identify cardinality ratios
 - 1 or Many?
 - Optional or Mandatory?
 - Produce an E/R model using those entities, attributes and relationships

Example 1: entities (nouns)

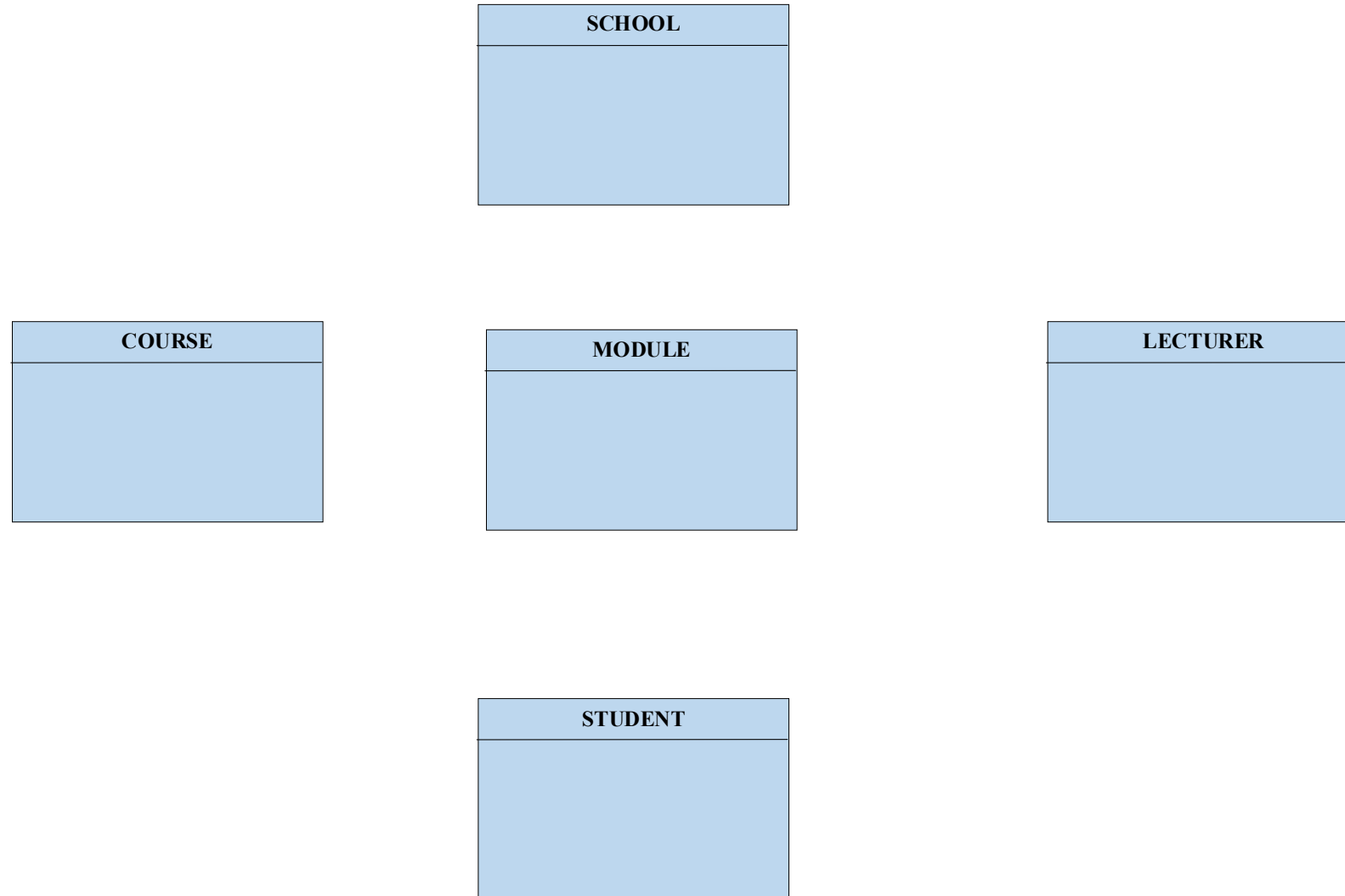
- A university consists of a number of **schools**. Each school offers several **courses**. A particular course could only be offered by a single school. A number of **modules** make up each course where one single module could be in several courses. **Students** enrol in a particular course at a time and take modules towards the completion of that course. Each module is taught by a **lecturer** from the appropriate school, and each lecturer tutors a group of students.
- Entities:
 - **School**
 - **Courses**
 - **Modules**
 - **Students**
 - **Lecturer**

Example 1: relationships (verbs)

- A university consists of a number of schools. Each school offers several courses. A particular course could only be offered by a single school. A number of modules make up each course where one single module could be in several courses. Students enrol in a particular course at a time and take modules towards the completion of that course. Each module is taught by a lecturer from the appropriate school, and each lecturer tutors a group of students.
- Relationships
 - Offers
 - make up
 - enrol in
 - Take
 - taught by
 - From
 - tutors

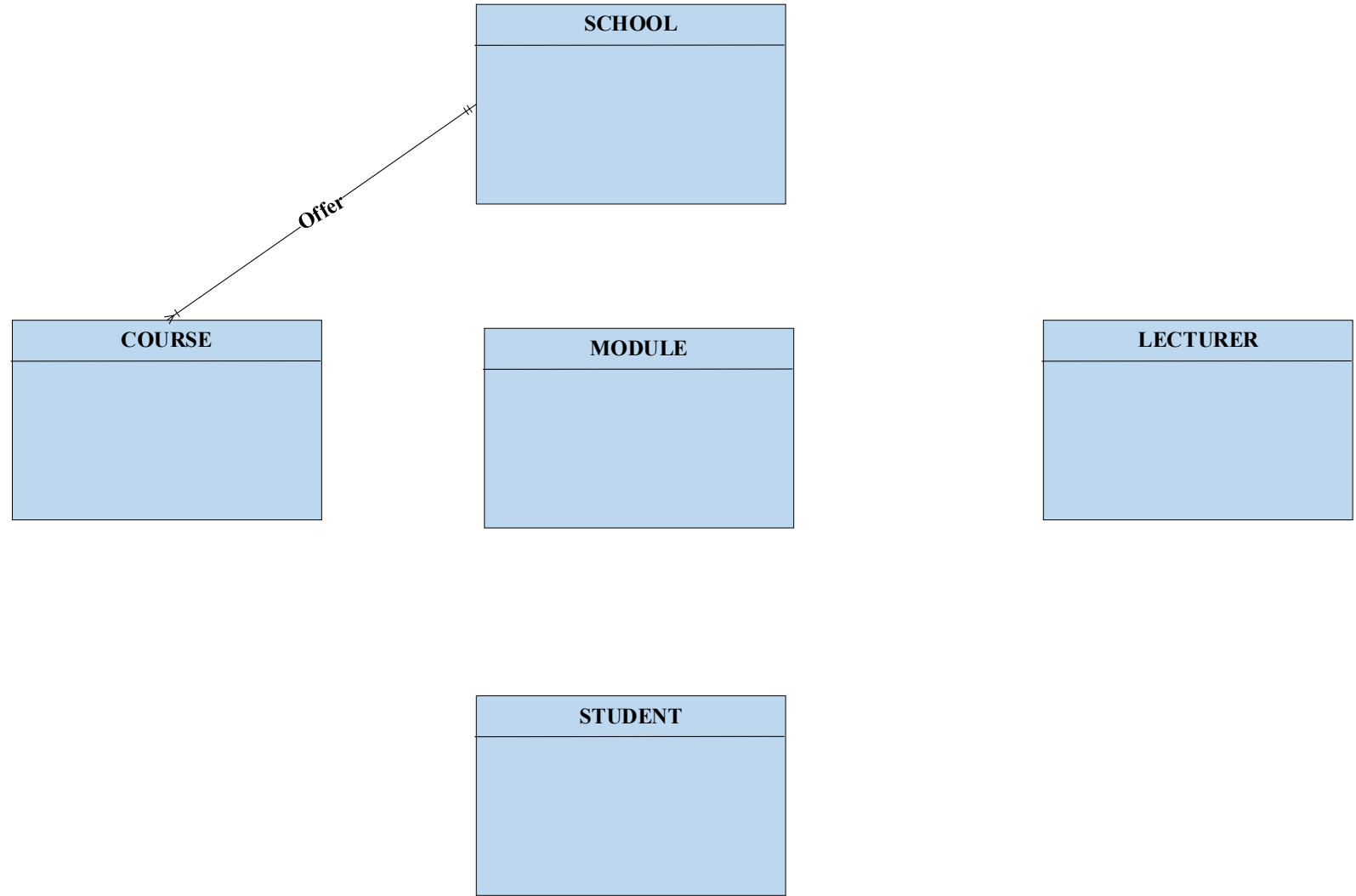
Example 1: E-R diagram w/M:M (1)

- Entities: school, course, module, lecturer, student



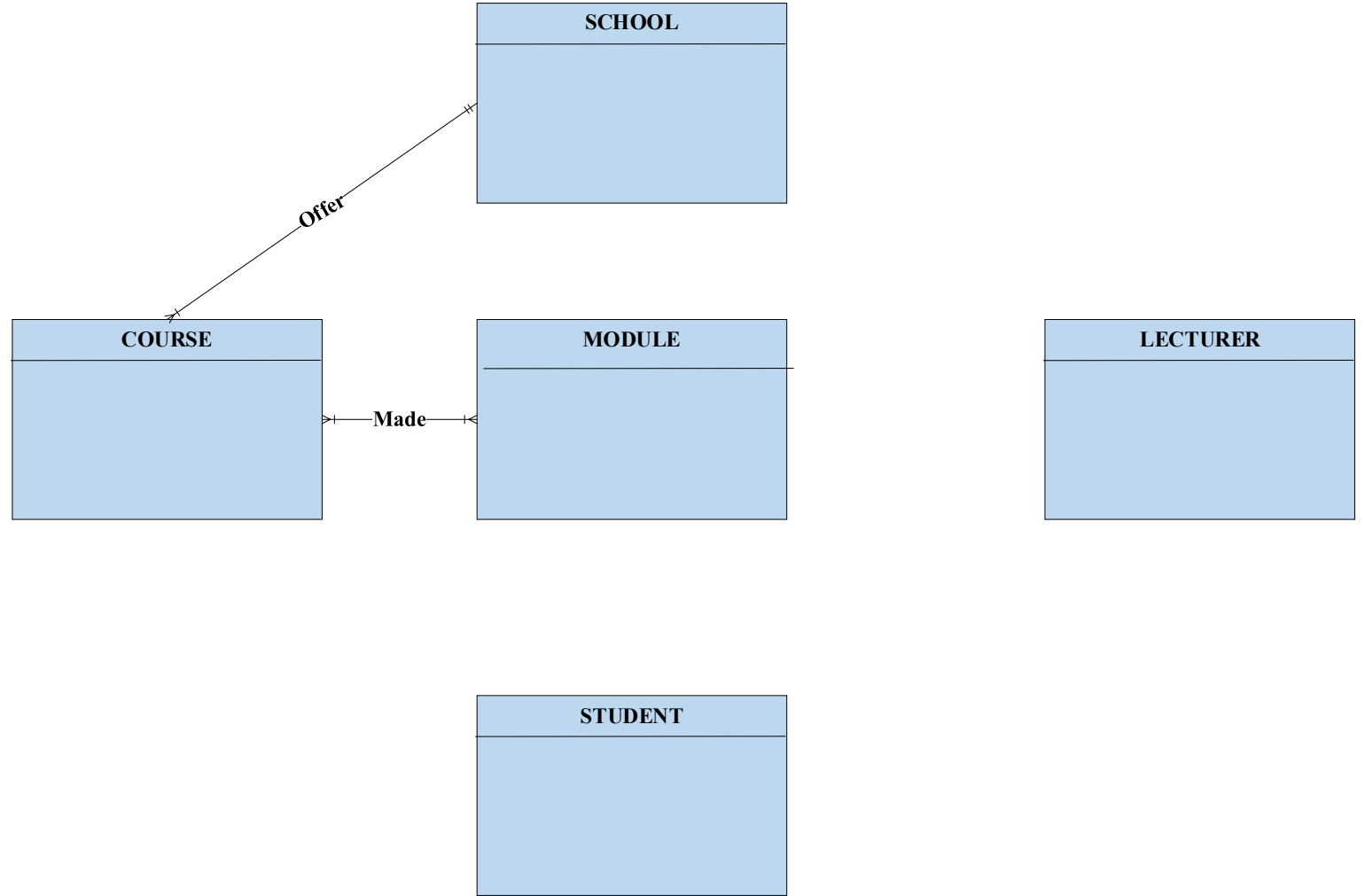
Example 1: E-R diagram w/M:M (2)

- Each **school** offers several **courses**. A particular course could only be offered by a single school.
- Assumption: A school can only exist if it is offering a course and a course can exist if it is offered by any school



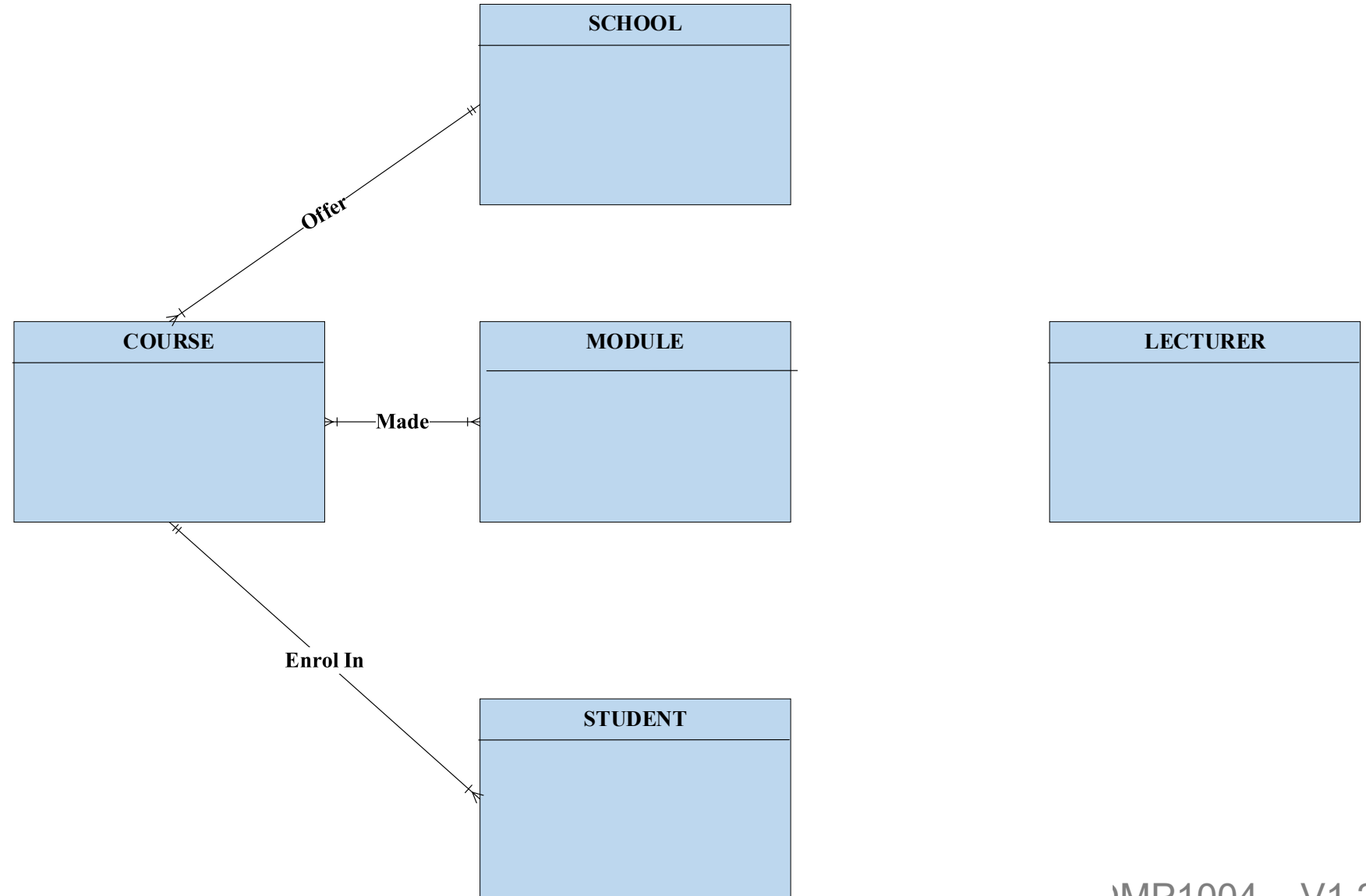
Example 1: E-R diagram w/M:M (3)

- A number of **modules** make up each **course** where one single module could be in several courses.



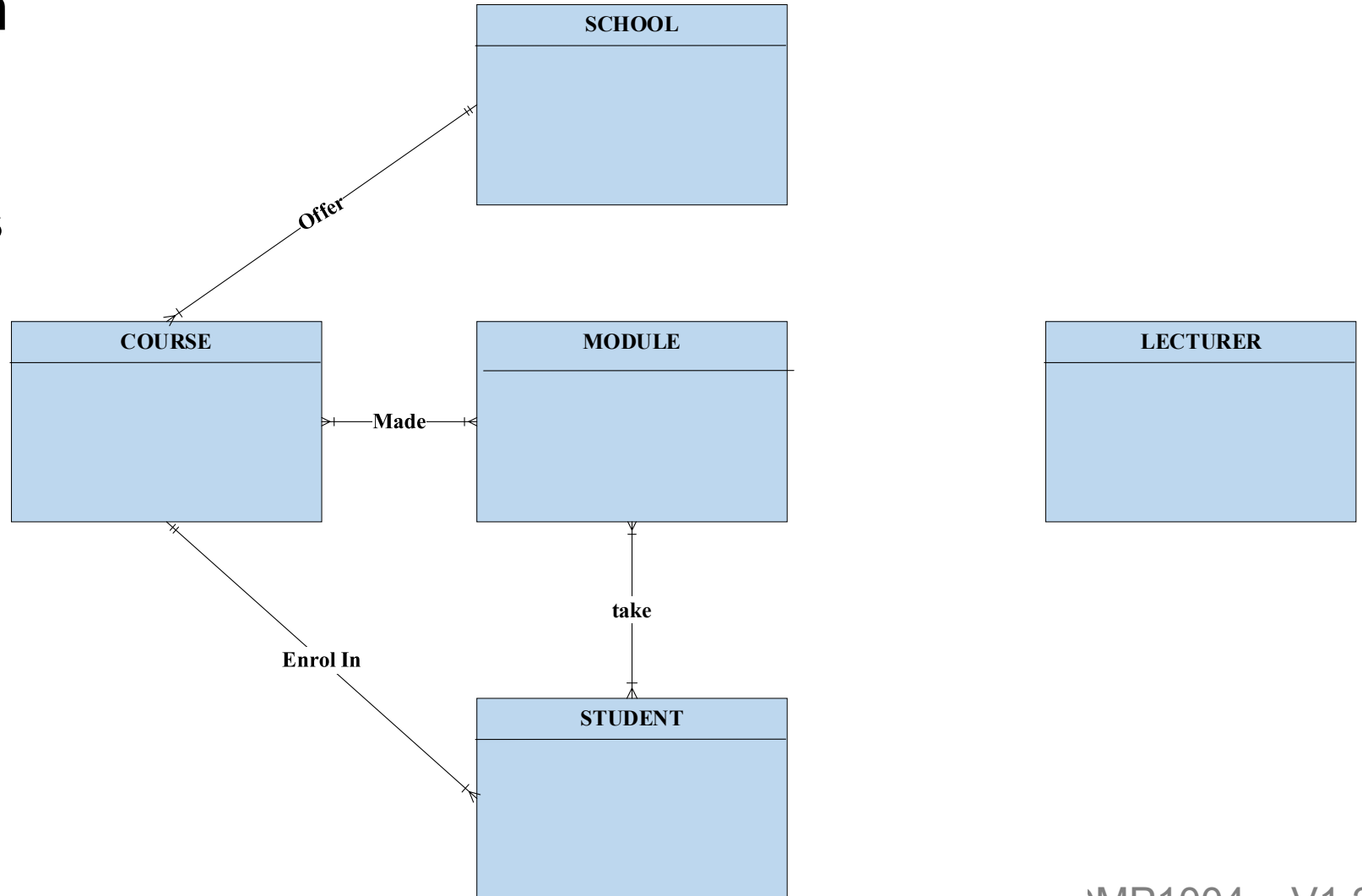
Example 1: E-R diagram w/M:M (4)

- **Students** enrol in a particular **course** at a time



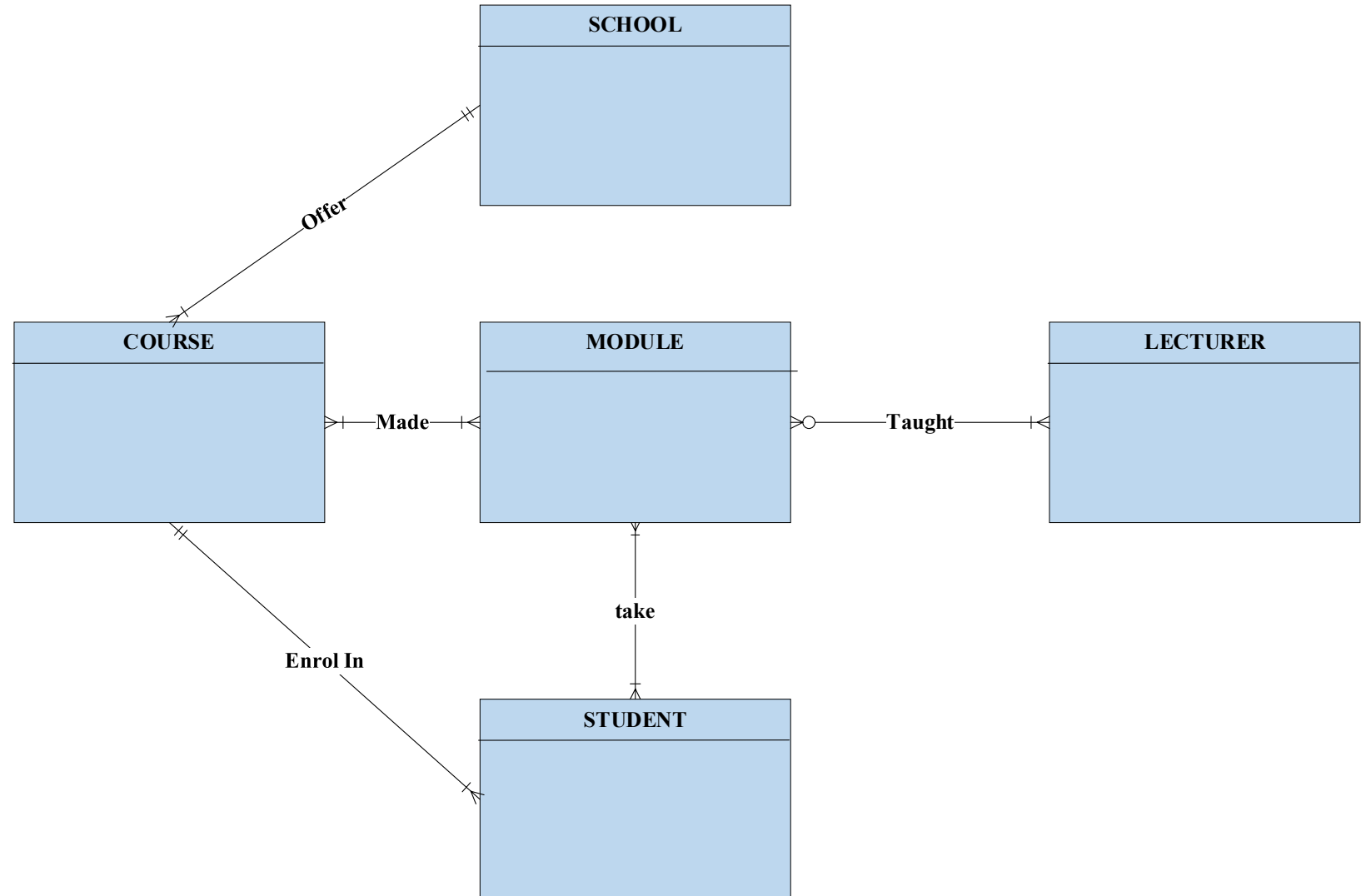
Example 1: E-R diagram w/M:M (5)

- **Student** take **modules** towards the completion of that course
 - Assumption
A module only exist if it is taken by at least one student



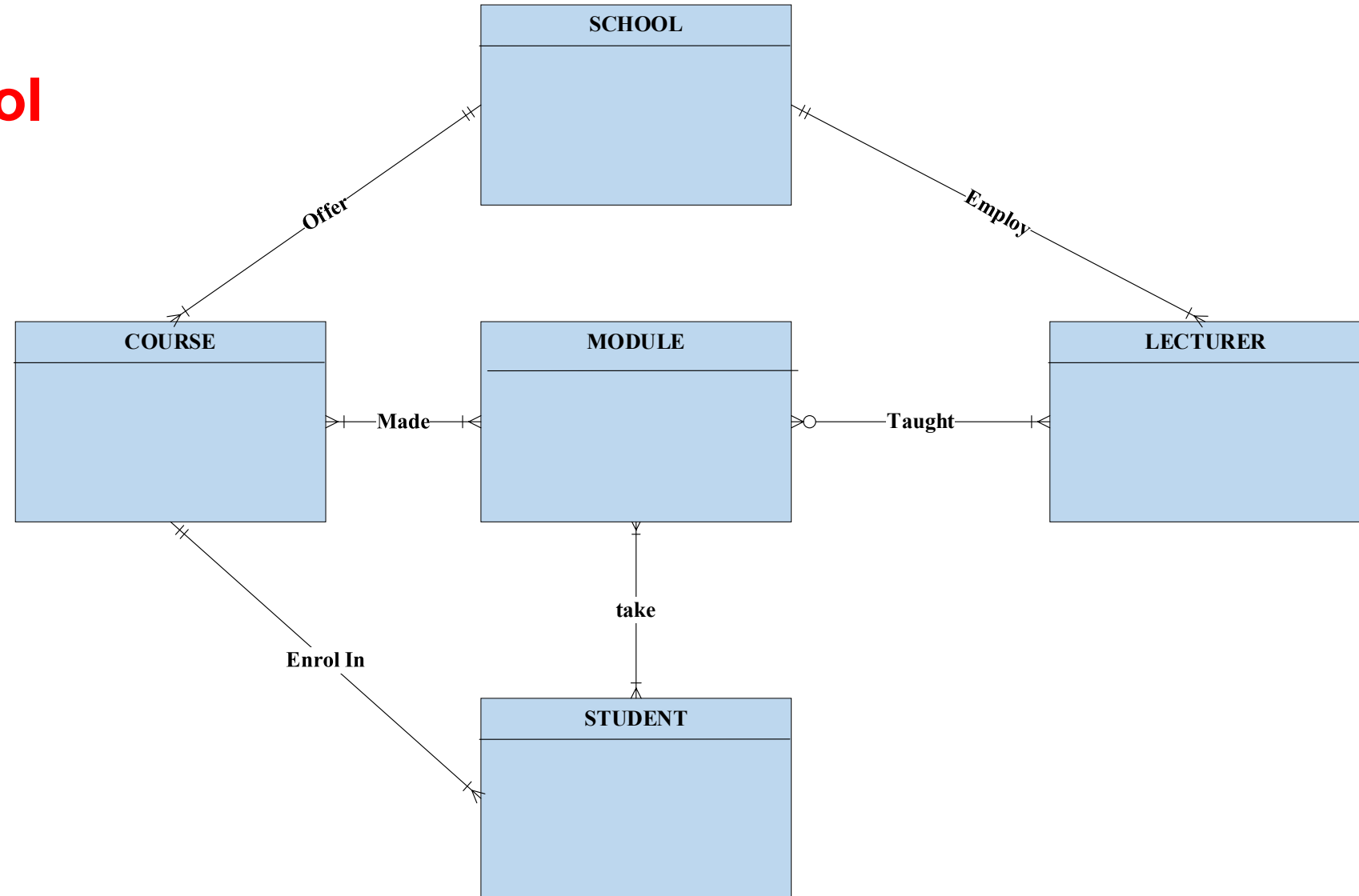
Example 1: E-R diagram w/M:M (6)

- All **modules** are **taught by** one or more **lecturers**
- Not all lecturers teach modules



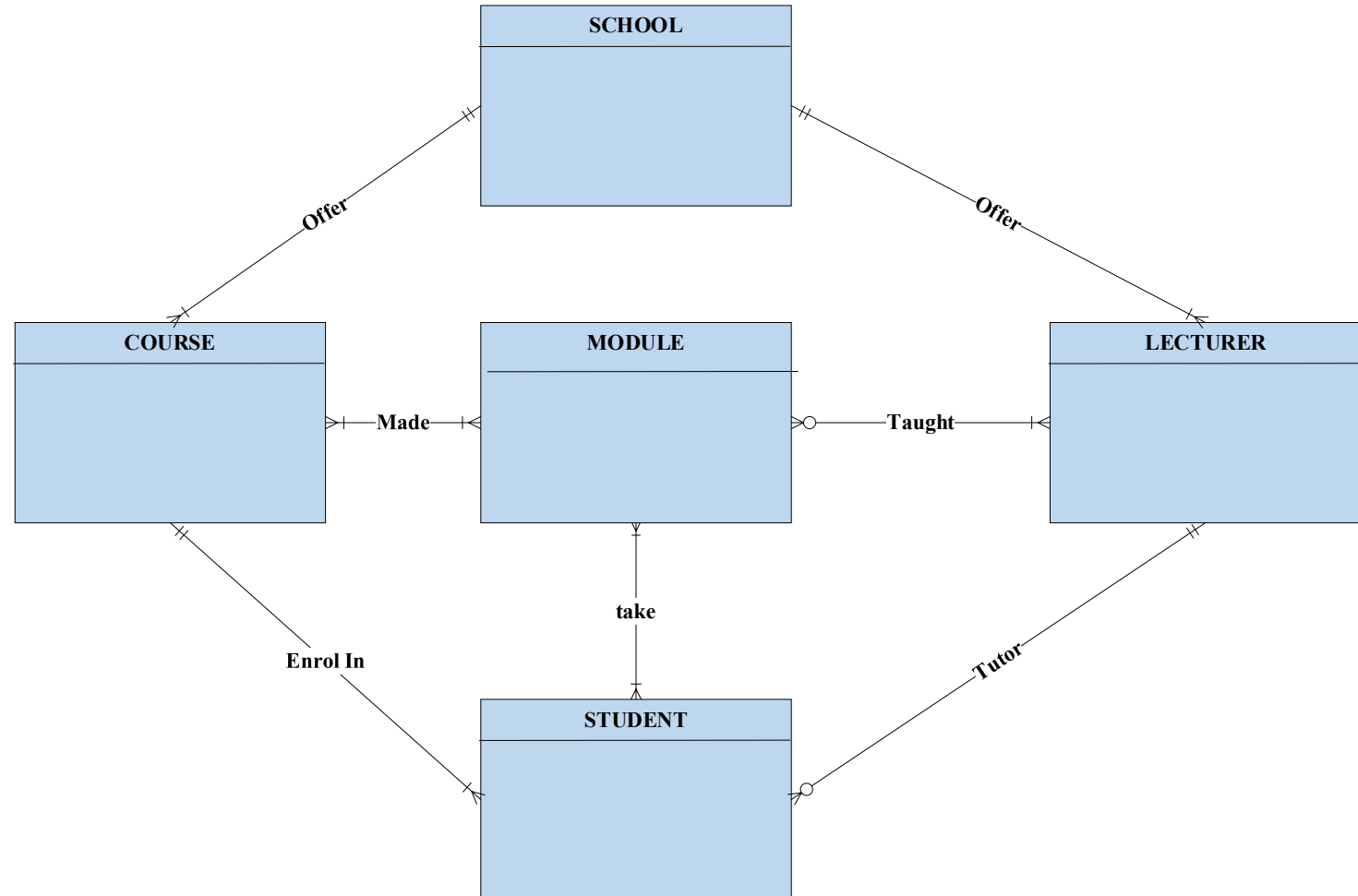
Example 1: E-R diagram w/M:M (7)

- All **lecturers** are employed by a **School**



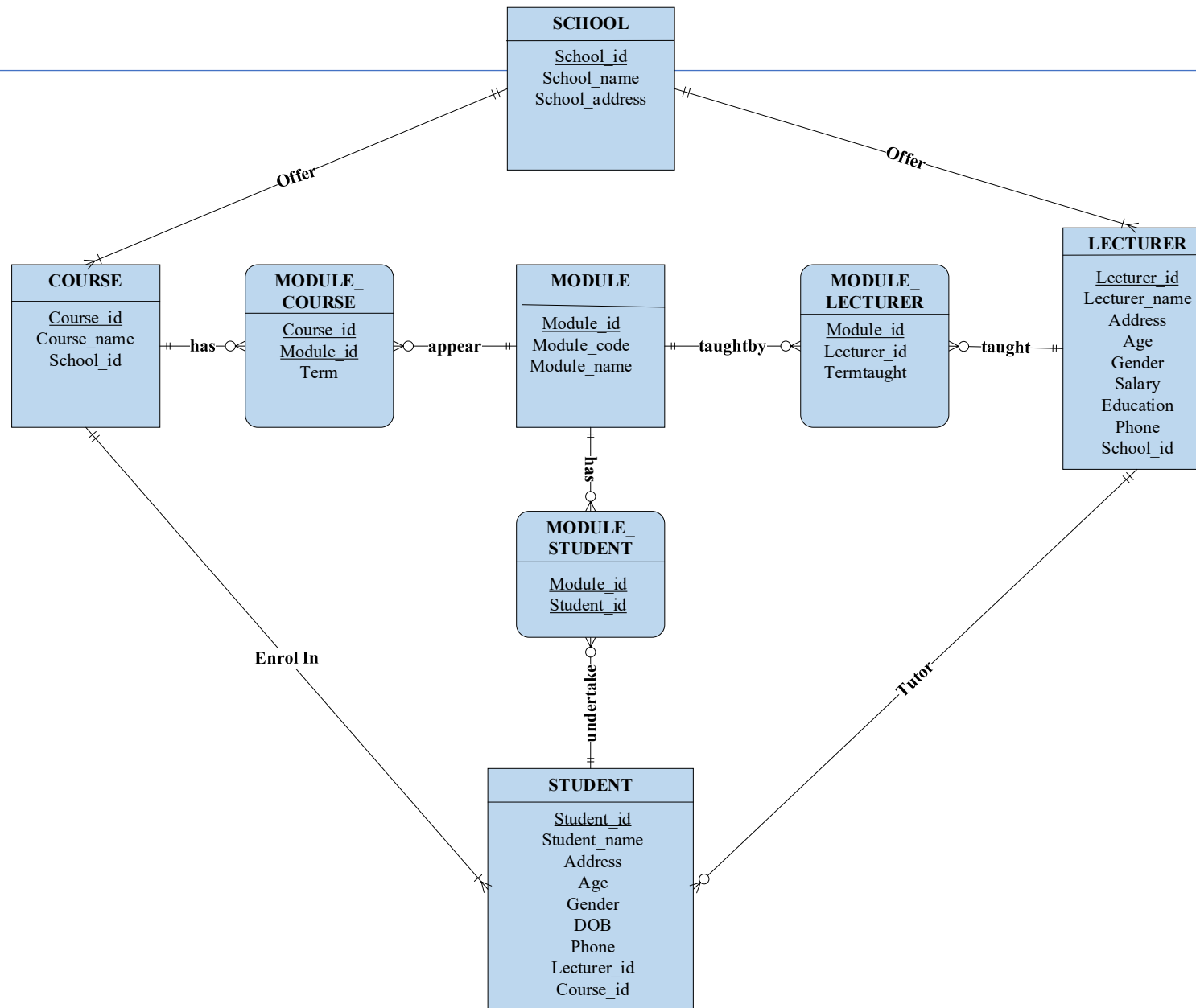
Example 1: E-R diagram w/M:M (8)

- Each **lecturer** **tutors** a group of **students**.
 - Assumption:
 - There might be some lecturer who are not tutors
 - Every student have one tutor



Example 1: E-R diagram w/M:M (complete)

- Identify many to many relationships
 - Course has many modules and module appear in many courses
 - Lecturer teach many modules and a module can be taught by many lecturers
 - A module can be taken by many students and a student can take many modules
- Break many to many relationship
- Add attributes



Summarising database design

- Analyse the real world
- Develop an initial E-R diagram
- Improve and debug the E-R diagram
- Implement the E-R diagram as tables, attributes and relationships (primary / foreign keys)

Summary

- E-R Modelling is about building a model of the real world
 - There are modelling choices at every point
- Entities / Attributes / Relationships
 - When we decide what these are we are deciding how to represent reality
 - Constructing boundaries, articulating relations between things, deciding what counts and what doesn't, how to categorise bits of the world as bits of database
- Interpreting a description of a situation, mapping the language out:
 - Nouns are entities(?)
 - Verbs are relationships(?)
- Deconstruct M:M cardinalities (break M:M relationships to use 1:1 or 1:M)
 - (NOTE: We've **not** always been doing this fully in these slides for speed!)
- Be aware of different notations